Documentation N8N + Automatisation TIKTOK :

Sommaire :

1 – Installation des dépendances

1.1 – Mise à jour des paquets
1.2 – Installation de FFMPEG
1.3 – Installation de Python3 et PIP
1.4 – Installation de YT-DLP et des cookies YouTube
1.5 – Installation de MariaDB + Création de la BDD
1.6 – Installation de NodeJS et NPM
1.7 – Installation de bgutil et configuration
1.8 – Installation de Deno
1.9 – Installation de Ngrok
1.10 – Installation de N8N
1.11 – Problèmes rencontrés et résolution de ceux-ci

2 – Automatiser TikTok avec n8n : de A à Z

2.1 – Mise en place de l'environnement n8n

2.2 – Configuration du Refresh Token et du token + Association du Compte TikTok cible

2.3 – Explication du workflow automatisation tiktok complet node par node

1 – Installation des dépendances :

1.1 – Mise à jour des paquets (obligatoire) :

```
sudo apt update && sudo apt upgrade -y
```

1.2 – Installation des paquets ffmpeg (nécessaire pour **couper les vidéos de 1m30** et **les stocker**) :

```
sudo apt install -y ffmpeg
```

1.3 – Installation de Python3 et Pip (pour yt-dlp) :

```
sudo apt install -y python3 python3-pip
```

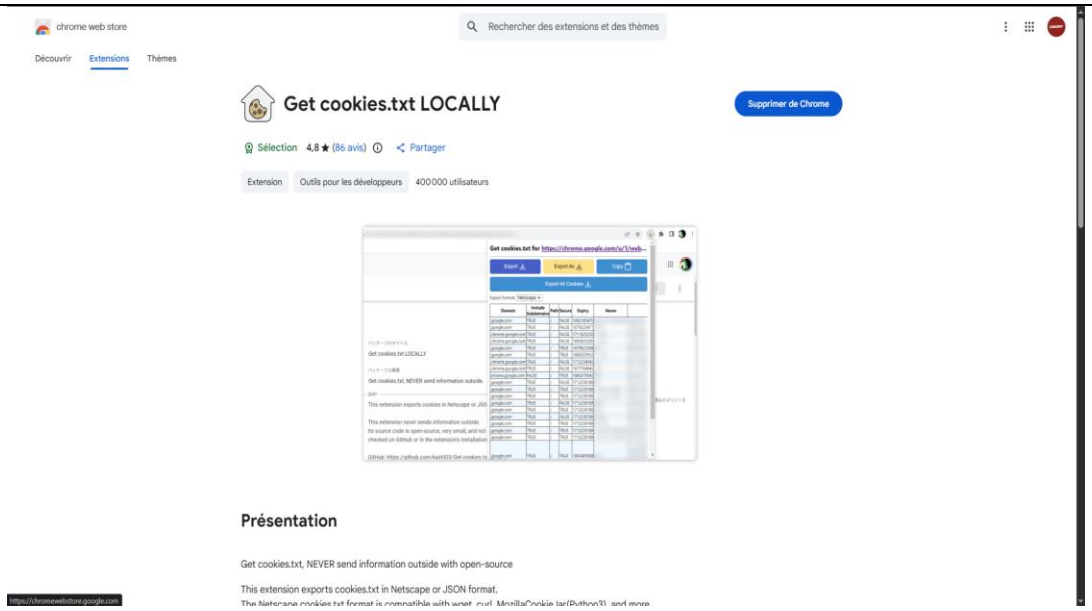
1.4 – Installation de yt-dlp et des cookies Youtube (téléchargement YouTube) :

Partie Linux:

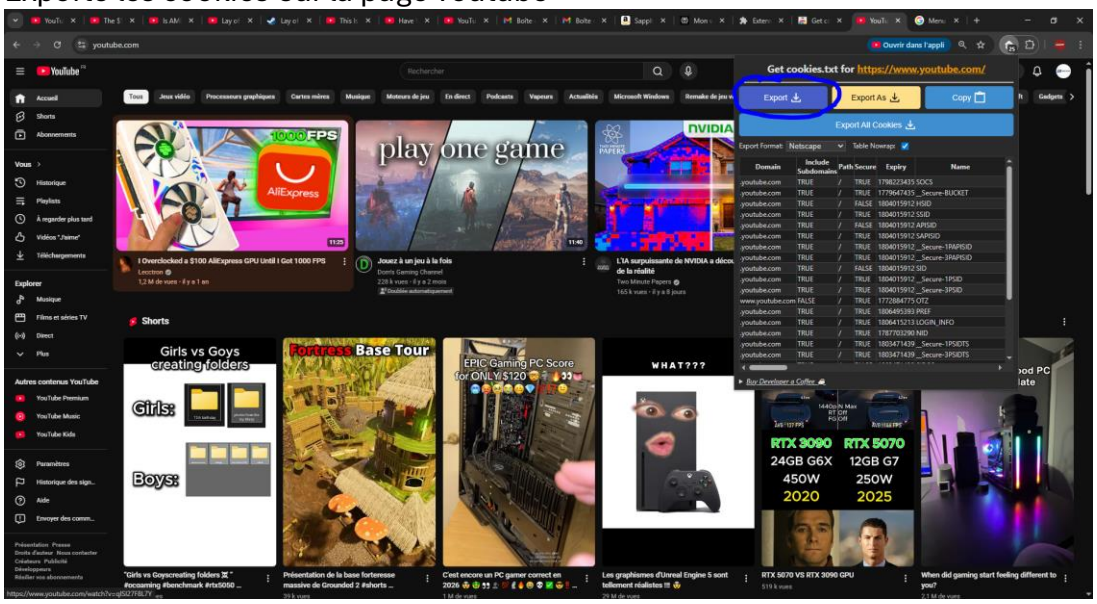
```
sudo curl -L https://github.com/yt-dlp/yt-dlp/releases/latest/download/yt-dlp  -o /usr/local/bin/yt-dlp

sudo chmod a+rx /usr/local/bin/yt-dlp
```

Partie Windows Exporter les cookies Youtube vers machine linux (A NOTER QUE LES COOKIES YOUTUBE EXPIRENT A ENVIRON 1 AN Donc il sera obligatoire de refaire la manipulation (Dans les logs de N8N il vous le dit EXPLICITEMENT → :
Requis : Compte GOOGLE

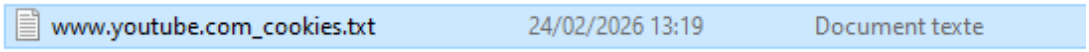
1. Sur le PC, Connectez-vous à youtube.com avec le compte Google
2. Installez l'extension «Get cookies.txt Locally comme ci-dessous



3. Exporte les cookies sur la page Youtube



4. Envoie le fichier sur le serveur OVH avec cette commande



Commande à entrer sur le CMD (Mode Administrateur) :
scp **CHEMIN DU FICHIER à changer en fonction de ou se situe le fichier**/cookies.txt **Nom d'utilisateur de la machine@IP de la machine distante**:/home/**Mettre le bon nom d'utilisateur de la machine distante/**youtube_cookies.txt

Vérification des versions de ffmpeg et yt-dlp :

```
ffmpeg -version

yt-dlp --version
```

1.5 – Installation + Création de la base de données + table SQL (Mariadb) :

```
sudo apt install mariadb-server

sudo mariadb-secure-installation (No No No No pour les 4 questions posées)

sudo mariadb

CREATE USER 'admin'@'localhost' IDENTIFIED BY 'Changer le mot de passe';
GRANT ALL PRIVILEGES ON *.* TO 'admin'@'localhost' WITH GRANT OPTION;
FLUSH PRIVILEGES;

mysql -u admin -p

CREATE DATABASE IF NOT EXISTS n8n_automations CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

use n8n_automations

CREATE TABLE video_queue (
   id INT AUTO_INCREMENT PRIMARY KEY,
   youtube_url VARCHAR(255) NOT NULL UNIQUE,
   video_title TEXT,
   video_description TEXT,
   video_tags TEXT,
   status VARCHAR(50) DEFAULT 'pending', -- pending, processing, completed, failed
   tiktok_publish_id VARCHAR(255),
   tiktok_share_url TEXT,
   error_message TEXT,
   created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

1.6 – Installation NodeJS et NPM et N8N :

```
sudo apt install curl
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash - sudo apt-get
install -y nodejs
```

1.7 – Installation bgutil et configuration :

```
git clone --single-branch --branch 1.2.2 https://github.com/Brainicism/bgutil-ytdlp-
pot-provider.git ~/bgutil

cd ~/bgutil/server && npm install && npx tsc
```

```
pip install bgutil-ytdlp-pot-provider

sudo nano /etc/systemd/system/bgutil.service

------ Contenu du fichier bgutil.service à insérer ----------------------------------------------
[Unit]
Description=bgutil yt-dlp PO Token Server
After=network.target

[Service]
ExecStart=/usr/bin/node /home/à changer avec le bon nom d'utilisateur de la machine linux/bgutil/server/build/main.js
Restart=always
RestartSec=5
User=operateur <--- à changer avec le bon nom d'utilisateur de la machine linux

[Install]
WantedBy=multi-user.target
```

Ctrl + S pour sauvegarder et CTRL + X pour quitter

```
sudo systemctl daemon-reload
sudo systemctl enable bgutil
sudo systemctl start bgutil
```

1.8 – Installation de Deno :

```
curl -fsSL https://deno.land/install.sh | sh

echo 'export DENO_INSTALL="/home/ à changer avec le bon nom d'utilisateur de la machine linux/.deno"' >> ~/.bashrc

echo 'export PATH="$DENO_INSTALL/bin:$PATH"' >> ~/.bashrc

source ~/.bashrc

deno –version

sudo ln -s /home/ nom d'utilisateur Linux/.deno/bin/deno /usr/local/bin/deno
```

1.9 – Installation de Ngrok :

```
curl -sSL https://ngrok-agent.s3.amazonaws.com/ngrok.asc | sudo tee /etc/apt/trusted.gpg.d/ngrok.asc >/dev/null
echo "deb https://ngrok-agent.s3.amazonaws.com buster main" | sudo tee /etc/apt/sources.list.d/ngrok.list
sudo apt update && sudo apt install ngrok
```

1.10 – Installation de N8N :

```
sudo npm install -g n8n
```

1.11 – Problèmes rencontrés et résolution de ceux-ci :

1. **Cookies sécurisés activés par défaut et Nodes bloqués par sécurité par N8N**

```
  GNU nano 8.4                                  /home/operateur/.bashrc
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
  if [ -f /usr/share/bash-completion/bash_completion ]; then
    . /usr/share/bash-completion/bash_completion
  elif [ -f /etc/bash_completion ]; then
    . /etc/bash_completion
  fi
fi

export N8N_SECURE_COOKIE=false
export NODES_EXCLUDE="[]"

export PATH="$PATH:$(npm bin -g)"

export PATH="$HOME/.npm-global/bin:$PATH"

^G Aide      ^O Écrire      ^F Chercher    ^K Couper      ^T Exécuter    ^C Emplacement M-U Annuler     M-A Marquer
```

Si vous rencontrez cette erreur lors du premier démarrage de n8n après son installation, procédez comme suit dans le terminal :

<u>1. Ouvrez le fichier de configuration bash :</u>

nano ~/.bashrc

<u>2. Tout en bas du fichier, ajoutez les deux lignes suivantes :</u>

- **export N8N_SECURE_COOKIE=false**
  et
- **export NODES_EXCLUDE="[]"**

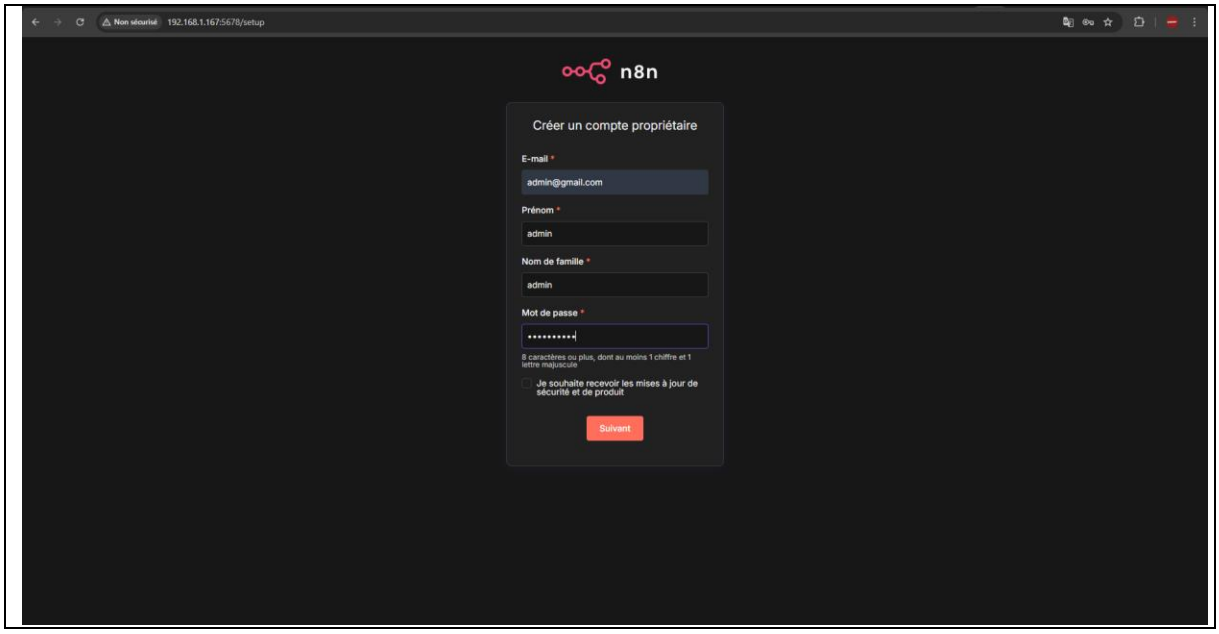
<u>3. Enregistrez avec CTRL + S, puis quittez avec CTRL + X.</u>

<u>4. Recharger le bash afin que les modifications prennent effet directement sans nécessiter un redémarrage du système</u> (Sauvegarde dans le système qui est appliqué à chaque redémarrage)

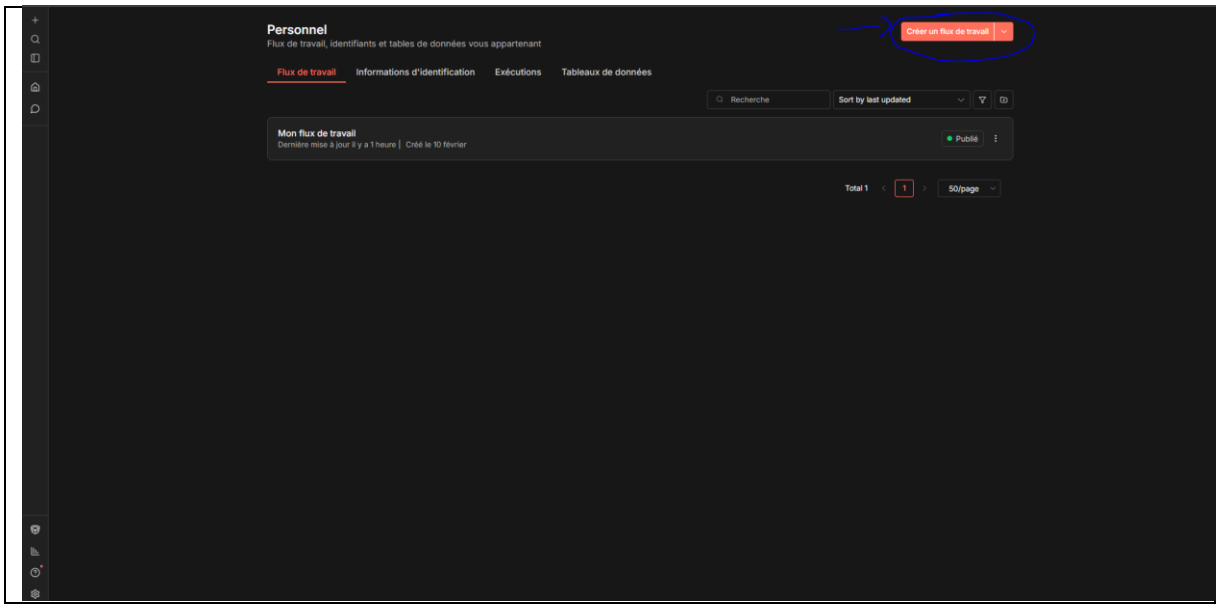
<u>Explication :</u> Cela permettra de **réactiver les nodes désactivés par N8N par sécurité** comme exemple le **Execute Command** et de **désactiver les cookies sécurisés qui nécessitent un certificat SSL**

<u>2 - Automatiser TikTok avec n8n : de A à Z :</u>

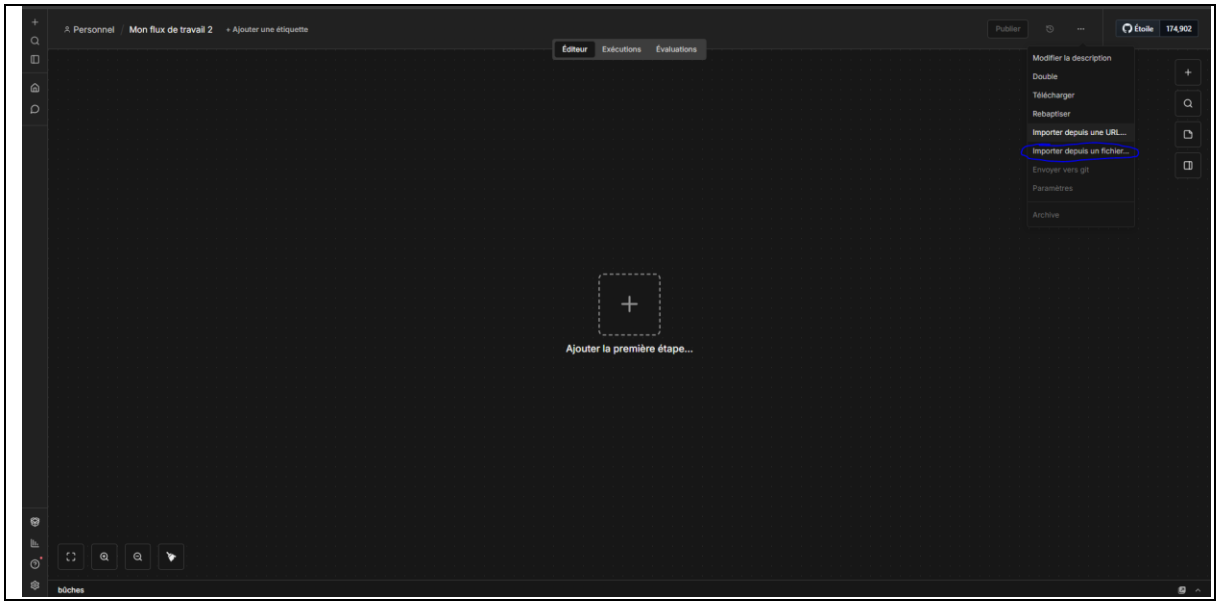
<u>2.1 – Mise en place de l'environnement n8n :</u>



Créer simplement votre compte (notez que c'est en **local** donc **des informations fausses ne feront rien et marcheront**) Ici c'est un **exemple** (mais réellement utilisé mais noté qu'un **mot de passe fort est fortement conseillé** même **si cela est en local**)

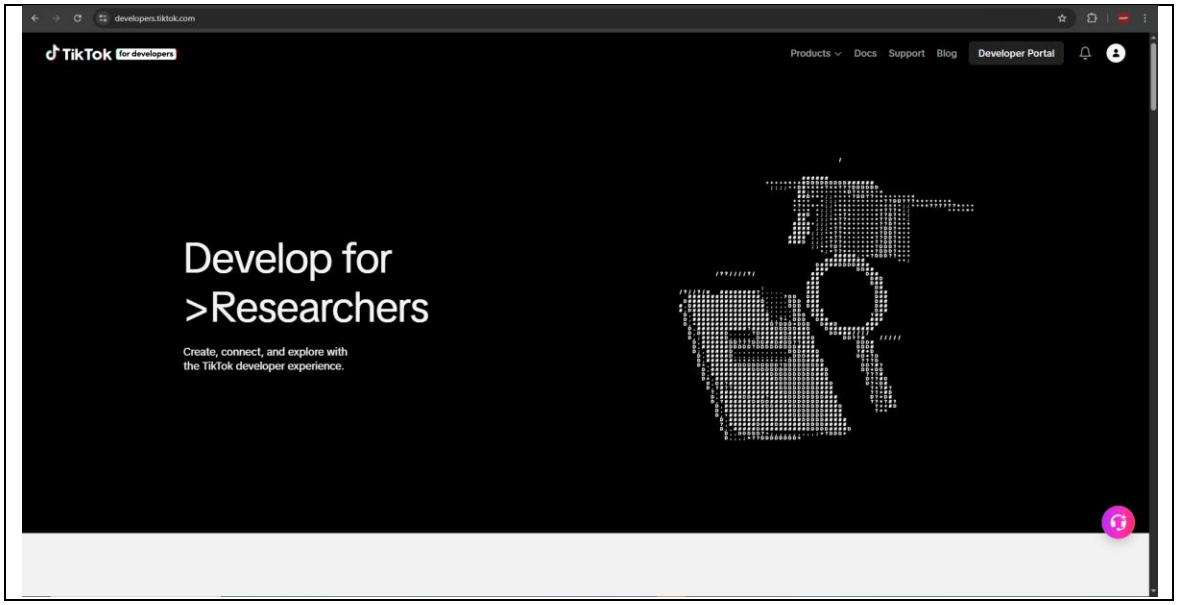


Cliquez sur « **Créer un flux de travail** » **en haut à droite**

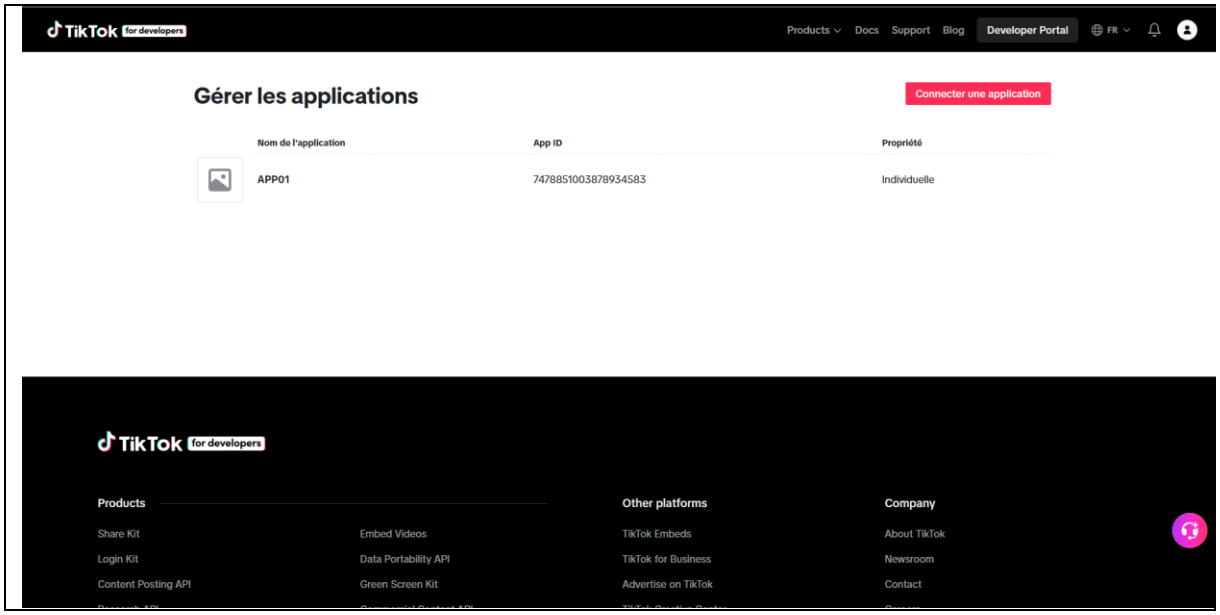


Cliquez sur les **3 petits points en haut à droite** et **sélectionner l'option « Importer depuis un fichier »** et **sélectionner ce fichier « automatisation_tiktok_workflow.json » et** n'oubliez pas après avoir importer le fichier **de publier le workflow**

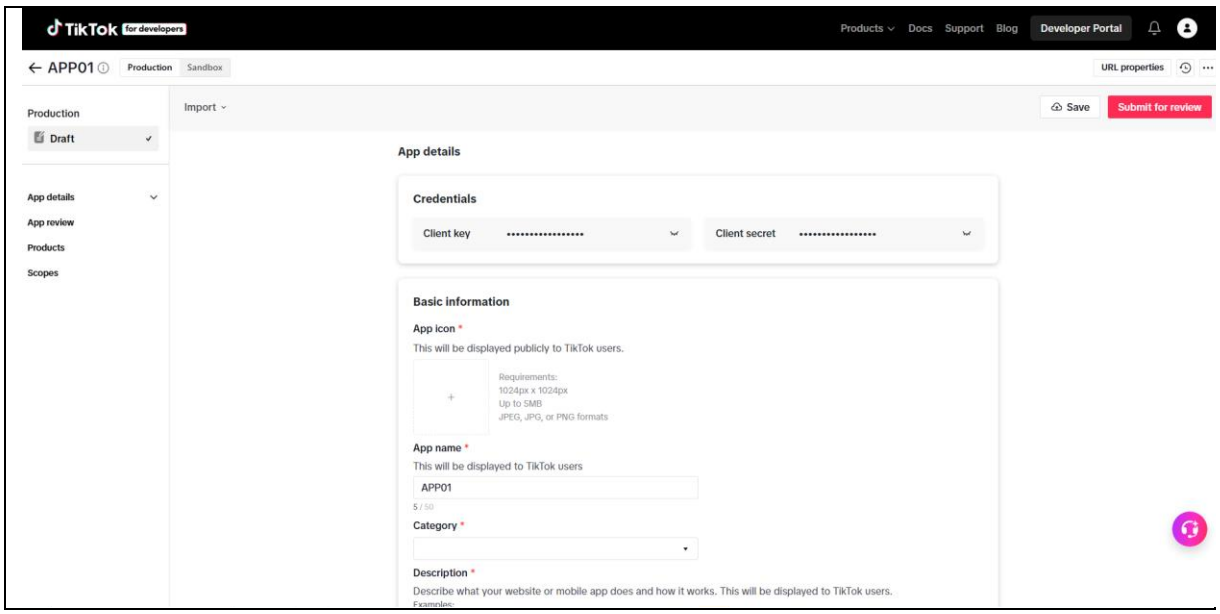
2.2 – Configuration du Refresh Token et du token (Token = 24 heures et Refresh Token= 365 jours) + Association du Compte TikTok cible (pour les vidéos qui seront publiées) :



1. Cliquez sur « **Developer Portal** » et ensuite sélectionner « **Manage apps** »



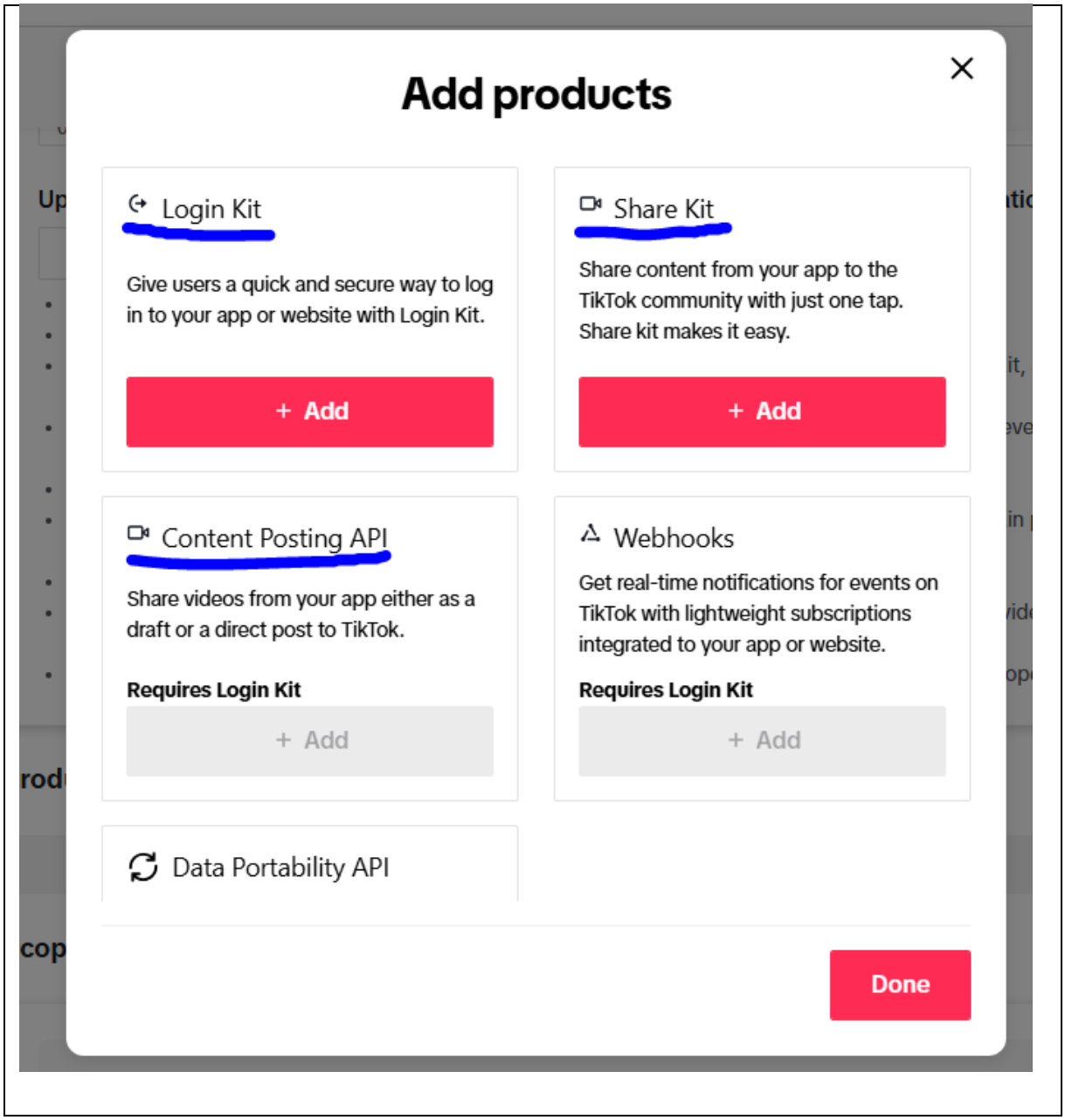
2. Cliquez sur l'application ici « **APP01** »

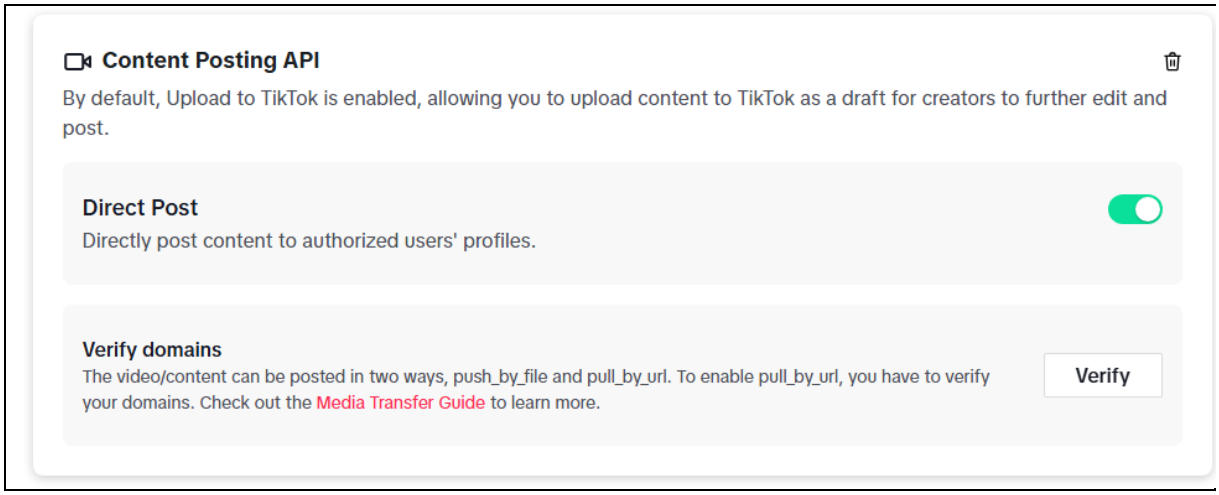


3. **Sélectionnez** le mode **Sandbox (Vidéos publiées privés)** ou **Production (Vidéos publiées publiques)** afin de pouvoir récupérer le **Client Key** et le **Client Secret** juste **en cliquant sur l'œil fermé des deux clés**

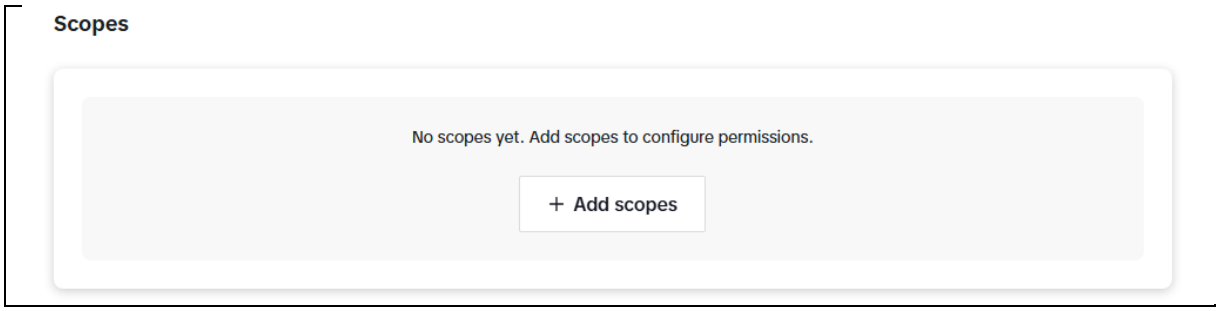
<u>PS :</u> Le **mode production est nécessaire pour publier des vidéos en public** alors que le **mode Sandbox est là pour les tests donc les vidéos**

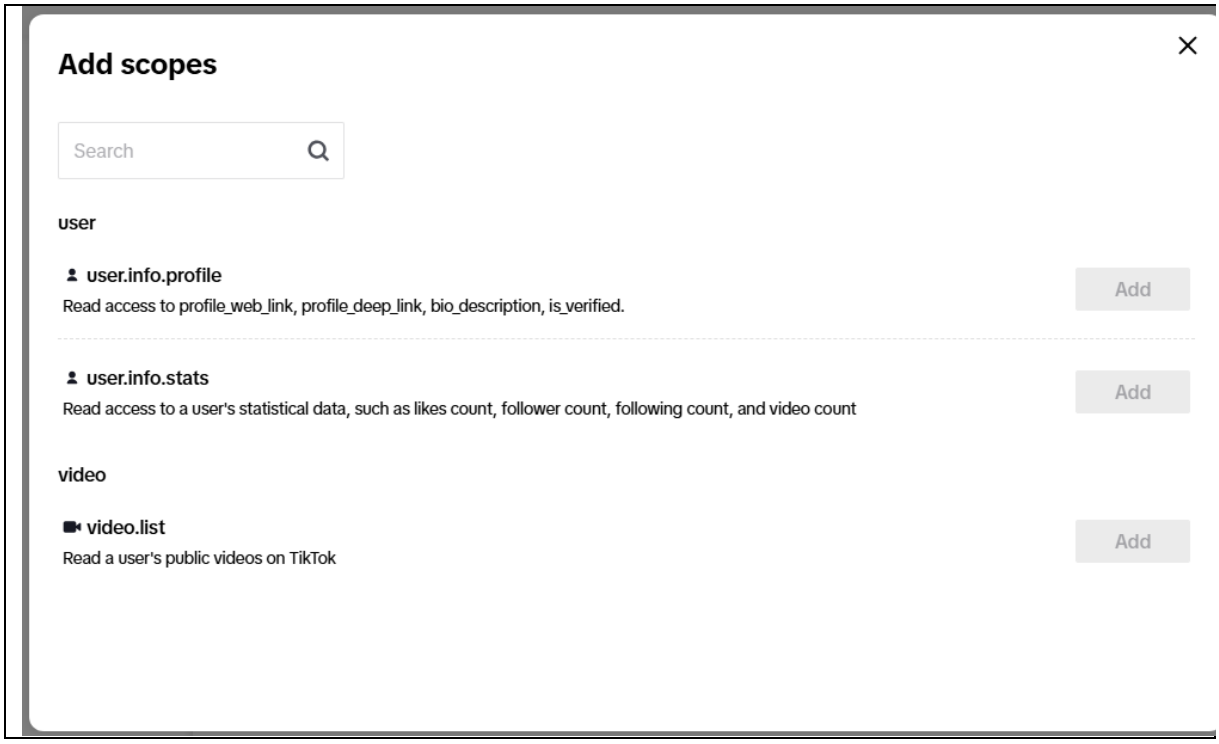
sont publiés en privé de plus **il est nécessaire de modifier une ligne coté workflow n8n afin de pouvoir publier en public** car **de base il est configuré pour publier en privé**



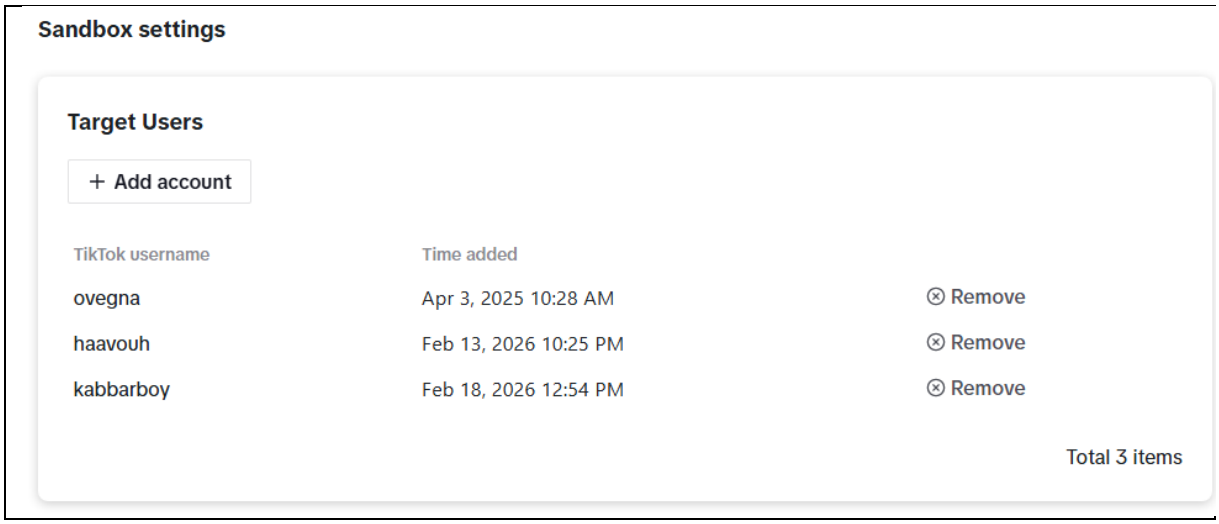


4. **Descendez** jusqu'à arriver à la catégorie « **Products** » ensuite **cliquez** sur « **+ Add products** » et ajoutez « **Login Kit** », « **Share Kit** », « **Content Posting API** ». De plus n'oubliez pas dès que vous sortez de la fenêtre dans la catégorie « **Content Posting API** » d'activer directement l'option « **Direct Post** »
   PS : N'oubliez jamais de faire « **Apply changes** » en **haut à droite** pour **appliquer les modifications**



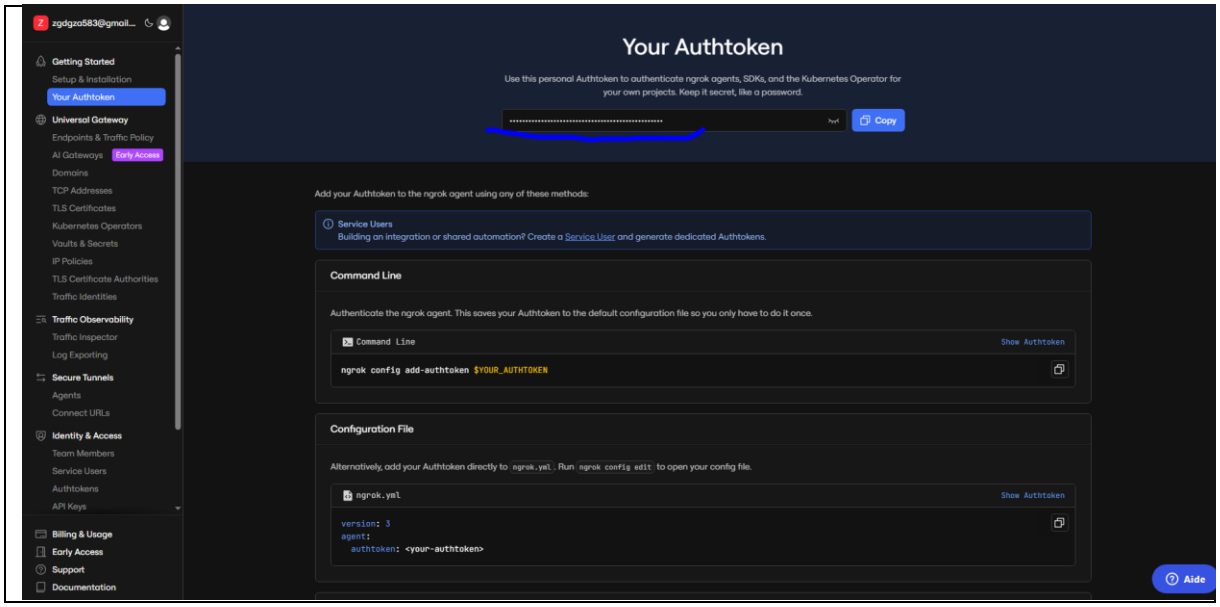


5. En dessous de la catégorie « **Products** », vous devriez voir la catégorie « **Scopes** », **cliquez** sur « **+ Add scopes** » et **ajoutez les trois scopes** des deux catégories « **user** » et « **video** »
   PS : N'oubliez jamais de faire « **Apply changes** » en **haut à droite** pour **appliquer les modifications**

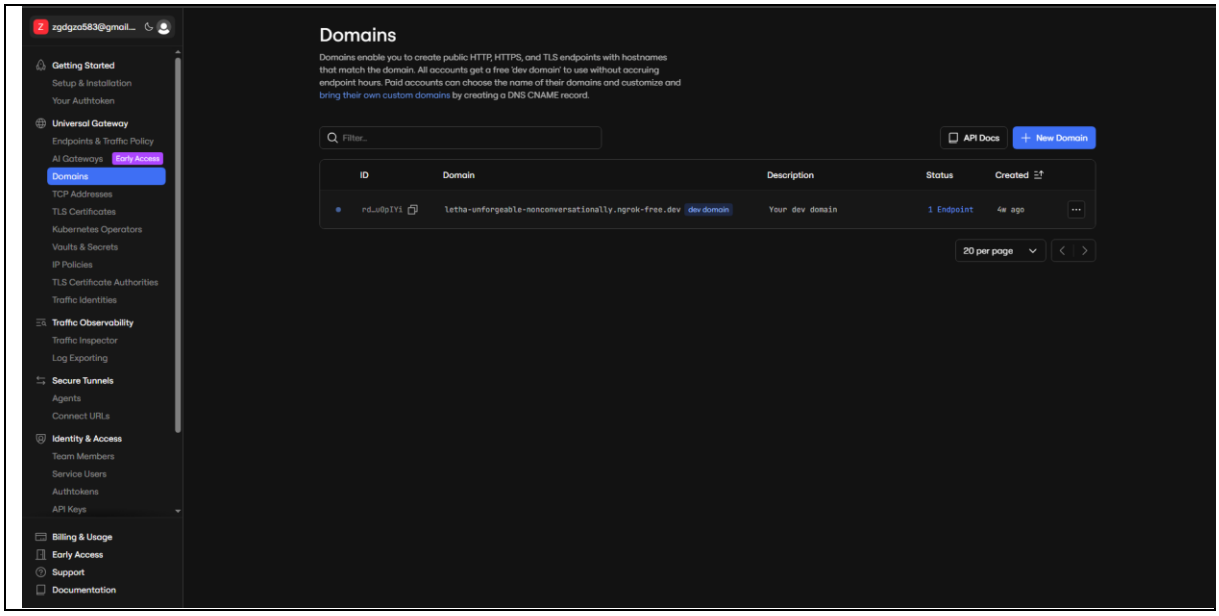


6. En mode « **Sandbox** » il est obligatoire **d'autoriser d'abord les comptes autorisés à publier du compte avant de passer par l'URL d'autorisation OAuth2**. C'est comme **une protection** que le mode « **Production** » **ne possède pas**
   <u>PS :</u> N'oubliez jamais de faire « **Apply changes** » en **haut à droite** pour **appliquer les modifications**

7. Allez sur le **site dashboard.ngrok.com** → **inscrivez-vous dessus**



8. Entrez cette commande dans le terminal linux : **ngrok config add-authtoken $YOUR_AUTHTOKEN** ← Remplacer par votre AuthToken



9. Lancer **ngrok** avec ces commandes (à faire une fois car persistant même après un redémarrage) car elle permet de lancer avec votre **nom de domaine définitive** (qui ne changera jamais) :
sudo nano /etc/systemd/system/ngrok.service

----------Contenu du fichier ngrok.service----------
[Unit]
Description=ngrok tunnel for n8n
After=network.target

[Service]
ExecStart=/usr/local/bin/ngrok http 5678 --domain=**Mettre le nom de domaine définitif** --log=stdout
Restart=always
RestartSec=5
User=**Mettre le bon utilisateur de la machine Linux**

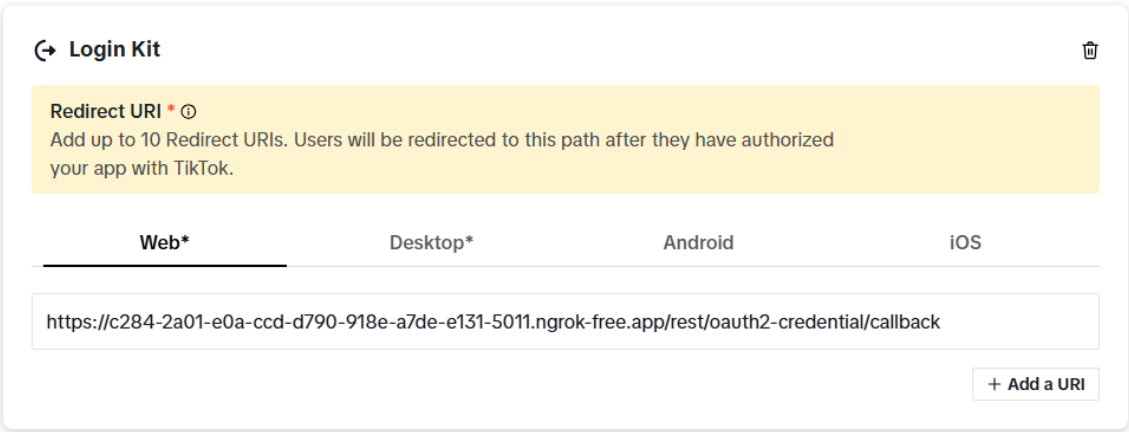
[Install]
WantedBy=multi-user.target

---

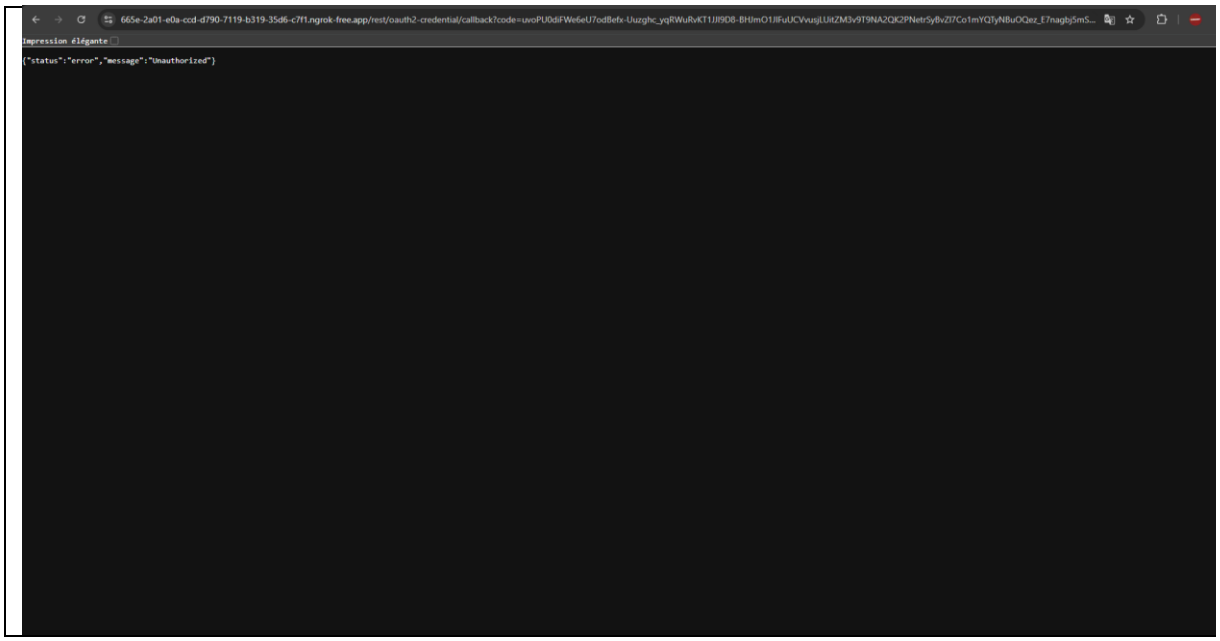
sudo systemctl daemon-reload
sudo systemctl enable ngrok
sudo systemctl restart ngrok

sudo systemctl status ngrok

PS : Pour avoir le nom de domaine définitive : Universal dashboard.ngrok.com→Gateway→Domains



10. Tu obtiens un lien de ce style : **https://c284-2a01-e0a-ccd-d790-918e-a7de-e131-5011.ngrok-free.app** ensuite il faut lui rajouter à la suite du lien ça : **/rest/oauth2-credential/callback** donc il devient comme ça **: https://c284-2a01-e0a-ccd-d790-918e-a7de-e131-5011.ngrok-free.app/rest/oauth2-credential/callback** et **le coller** sur le « **Login Kit** »
PS : N'oubliez jamais de faire « **Apply changes** » en **haut à droite** pour **appliquer les modifications**



<u>PS :</u> Il faut **aller très vite** car dès que vous avez le message : **{"status":"error","message":"Unauthorized"}** après avoir cliqué sur le lien**, le compteur de 10 minutes commence après ces 10 minutes le Refresh Token ne sera plus valable** et **vous devriez refaire la méthode à partir de l'étape 11**

11. Récupérons le lien construit sur l'étape 10, ensuite simplement il faudra prendre ce lien :
https://www.tiktok.com/v2/auth/authorize/?client_key=**$Clientkey àremplacer**&scope=video.upload,video.publish,user.info.basic&response_type=code&redirect_uri=**$Liencréeàl'étape10**state=random123
Vous aurez ce message si le lien est bon avec écrit :
{"status":"error","message":"Unauthorized"}
Il faudra récupérer le code dans le lien :
https://665e-2a01-e0a-ccd-d790-7119-b319-35d6-c7f1.ngrok-free.app/rest/oauth2-credential/callback?code=**uvoPU0diFWe6eU7odBefx-Uuzghc_yqRWuRvKT1JJl9D8-BHJmO1JlFuUCVvusjLUitZM3v9T9NA2QK2PNetrSyBvZl7Co1mYQTyNBuOQez_E7nagbj5mSgY9Jg-ioScxiUICCT02m75Q5yC5q5XEUu__8-lAuHl7rXhnjohMueHKbq7lxoaV3VBUCnHcEUD-tSZt71qIT3P809hWlVPInDhJTCkRkhqhr2YHl2Y-DqGOVmPRqrvQNNJYGAeX3kugee6P-kZPU3VTUtYaaq3V75nmOZfRM3F5drY14BchpdxNb8MvSZN5iUICt7ss9gUnOP90UKvpWYwZ06H%2Av%214921.e1**&scopes=user.info.basic%2Cvideo.publish%2Cvideo.upload&state=random123
Ici le code commence après « **code=** » **et finit avant «&scopes »**

12. **Commande à entrer dans le terminal rapidement car expiration du code sous 10 minutes :**

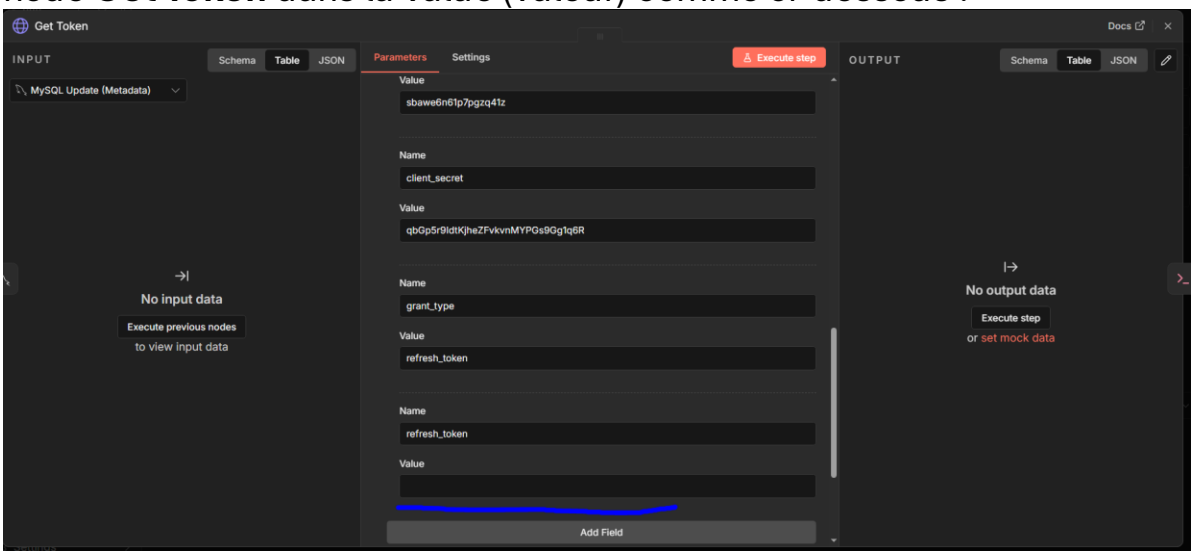
curl -X POST 'https://open.tiktokapis.com/v2/oauth/token/' -H 'Content-Type: application/x-www-form-urlencoded' -d 'client_key=**Votre Client Key'** -d 'client_secret=**Votre Client Secret**' -d 'code=**Le code obtenu à l'étape 11'** -d 'grant_type=authorization_code' -d 'redirect_uri=**Mettre le**

nom de domaine définitive gratuit fourni par ngrok/rest/oauth2-credential/callback'

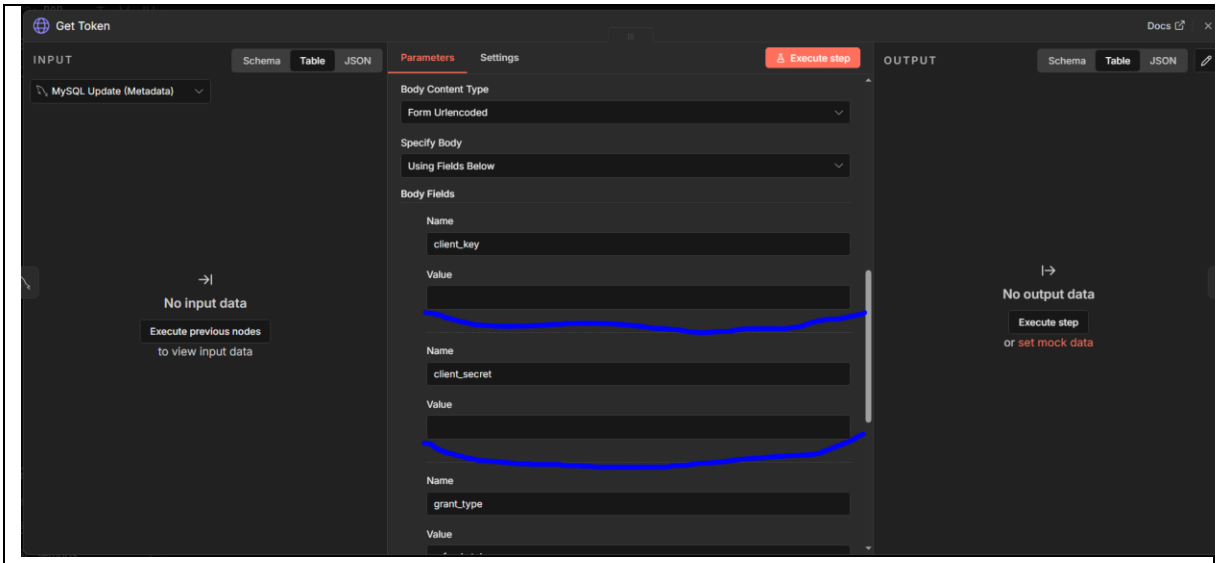
Résultat attendu :

{
 "access_token": "act.abc...",
 "expires_in": 86400,
 "open_id": "...",
 "refresh_token": "rft.def...",
 "refresh_expires_in": 31536000,
 "scope": "user.info.basic,video.publish,video.upload",
 "token_type": "Bearer"
}

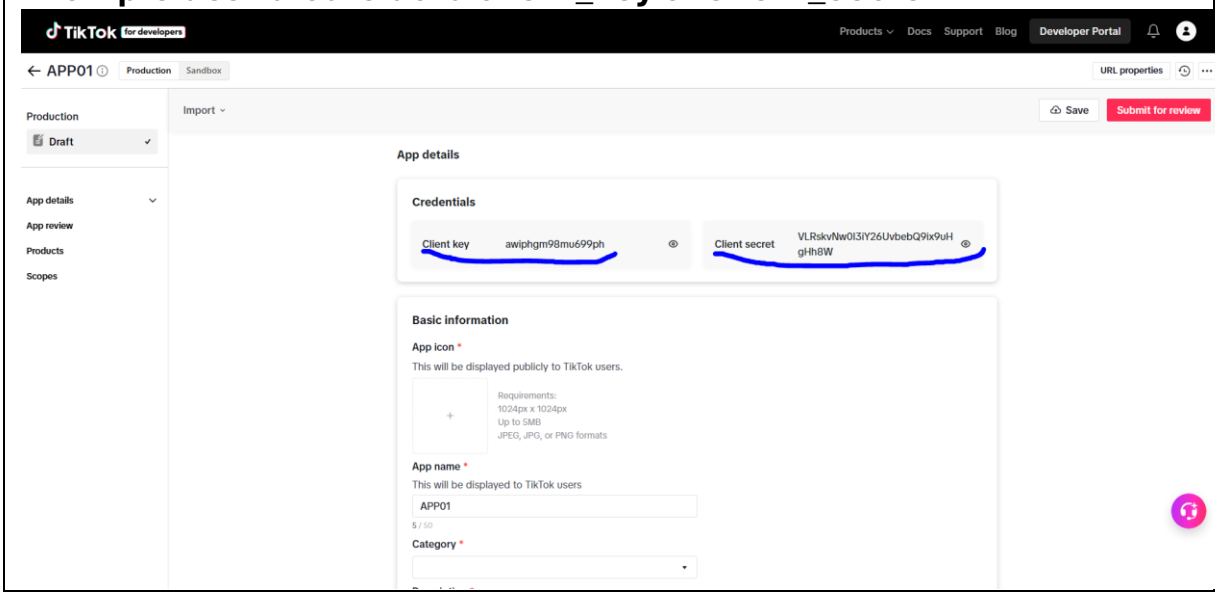
Une fois cela fait il faudra récupérer le **refresh_token** et l'intégrer dans le node **Get Token** dans la Value (valeur) comme ci-dessous :



N'oubliez pas aussi de remplir les valeurs de la **client_key** et **client_secret** trouvable sur votre **application Tiktok** sur le site de **developpers tiktok** : https://developers.tiktok.com/

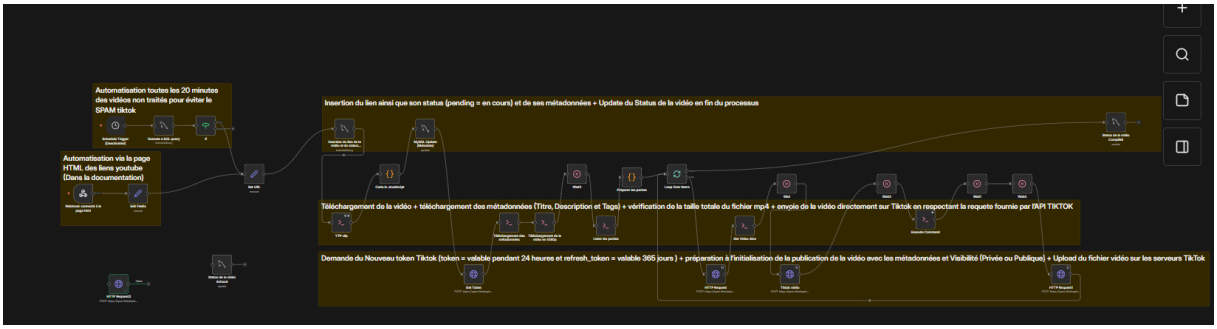


Exemple des valeurs de la **client_key** et **Client_secret** :

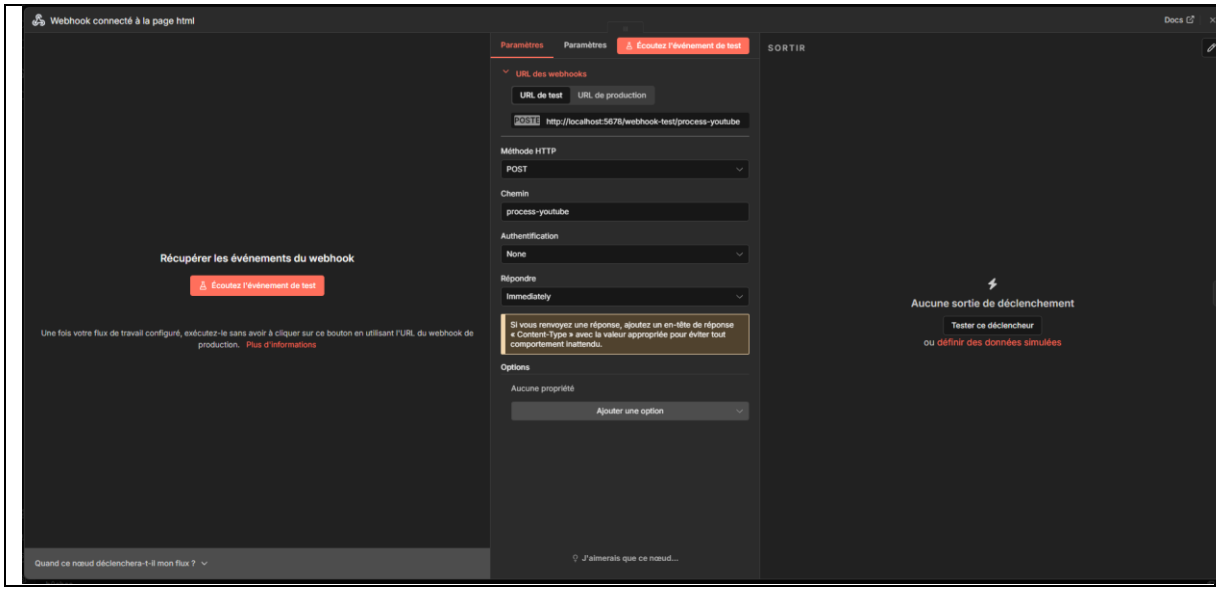


2.3 – Explication du workflow automatisation tiktok complet node par node :

Capture de l'ensemble du WORKFLOW :



<u>NODE 1 :</u> Webhook essentiel pour récupérer le lien de la vidéo



Code HTML:

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>YouTube → TikTok</title>
</head>
<body style="font-family:Arial;padding:50px;text-align:center;background:#f0f2f5;">
  <h1>🎬 YouTube → TikTok</h1>
  <form action="http://192.168.1.167:5678/webhook/process-youtube" method="POST" style="max-width:600px;margin:auto;">
    <input type="url" name="youtube_url" placeholder="Lien YouTube..." required
    style="width:100%;padding:15px;margin-bottom:10px;">
    <button type="submit" style="padding:15px 30px;cursor:pointer;">LANCER</button>
  </form>
</body>
</html>
```

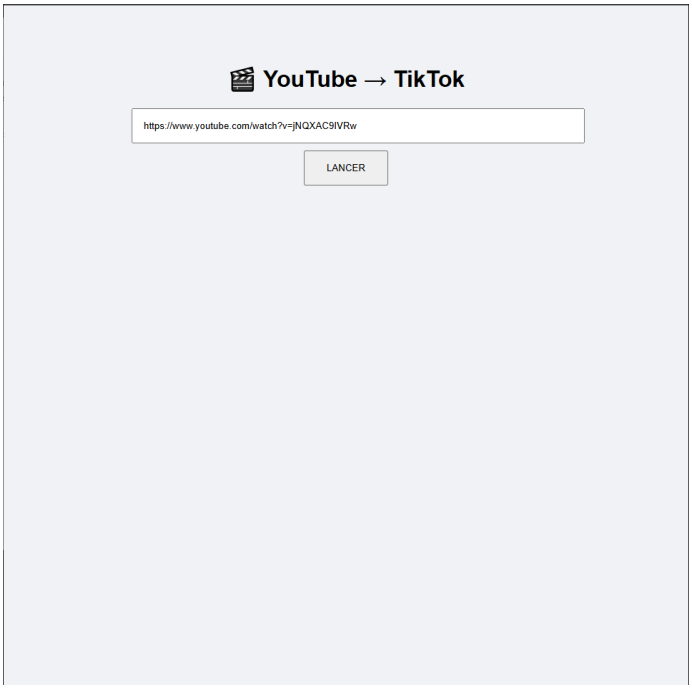
Explication du node :

Ce **nœud** reçoit la requête **POST** envoyée par la **page HTML**.

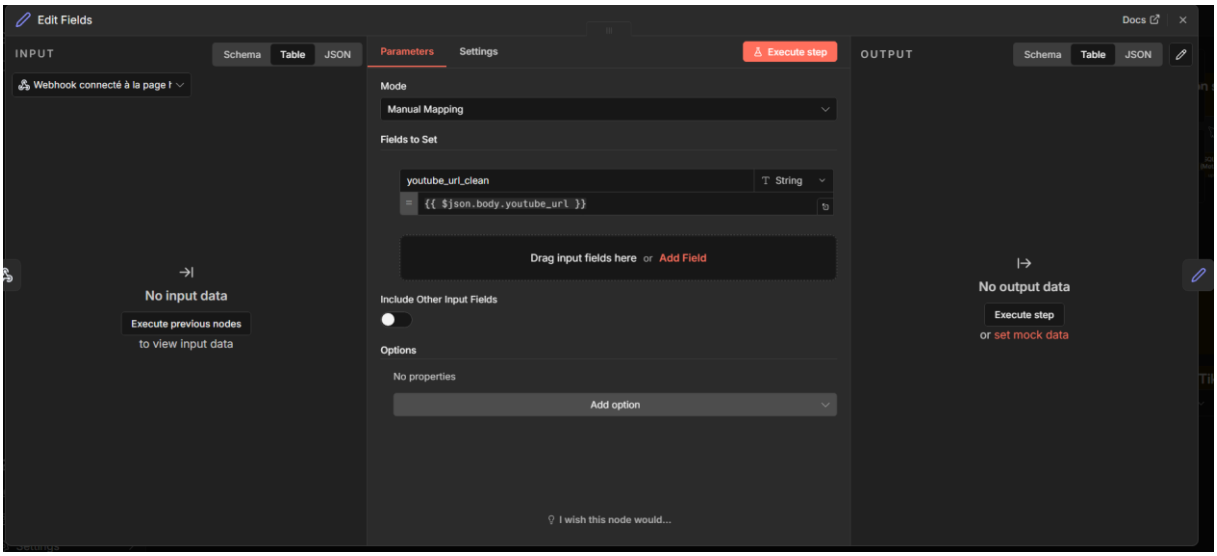
Cette requête contient l'**URL de la vidéo YouTube** à **télécharger** et à **enregistrer dans la base de données créée au préalable**.

Remarque : il faudra **modifier l'adresse IP** indiquée dans le **code HTML** pour qu'elle corresponde à celle de la machine sur laquelle est installé **n8n**.

- Si le fichier HTML est exécuté depuis **un autre ordinateur**, indiquez **l'adresse IP locale** de la machine hébergeant **n8n**.
- Si le fichier est utilisé sur **le même ordinateur**, remplacez l'adresse IP par **localhost:5678**.



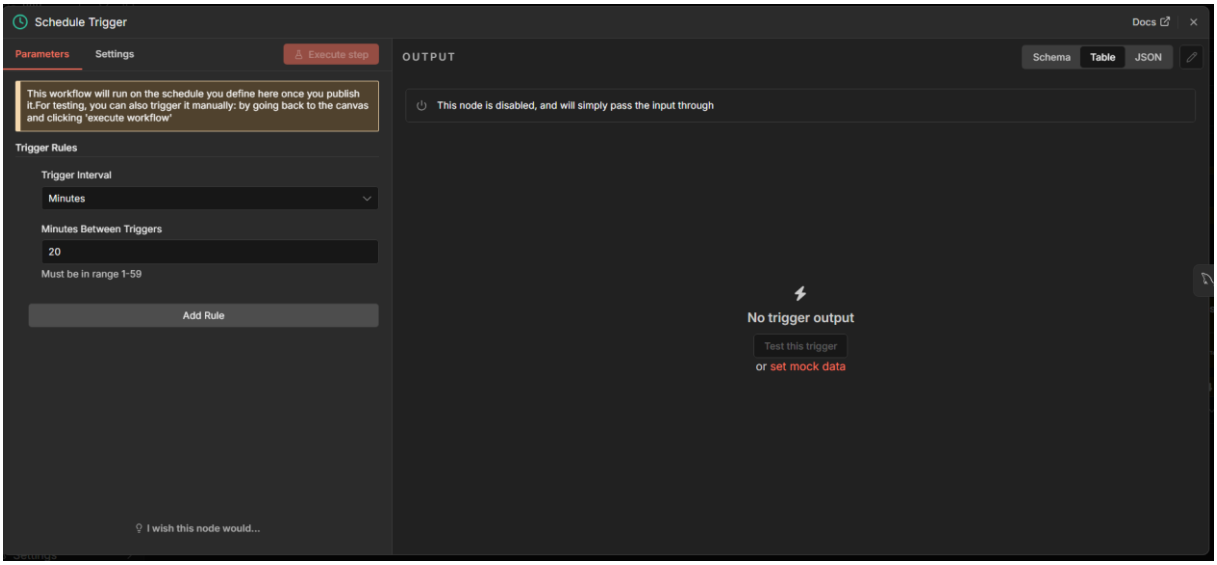
NODE 2 : Edit Fields



Explication du node :

- **Fonction :** Extrait youtube_url du body webhook
- **Paramètres clés :** Set youtube_url_clean = $json.body.youtube_url
- **Entrée :** Webhook body
- **Sortie :** Set URL
- **Autres :** Simple assign string.

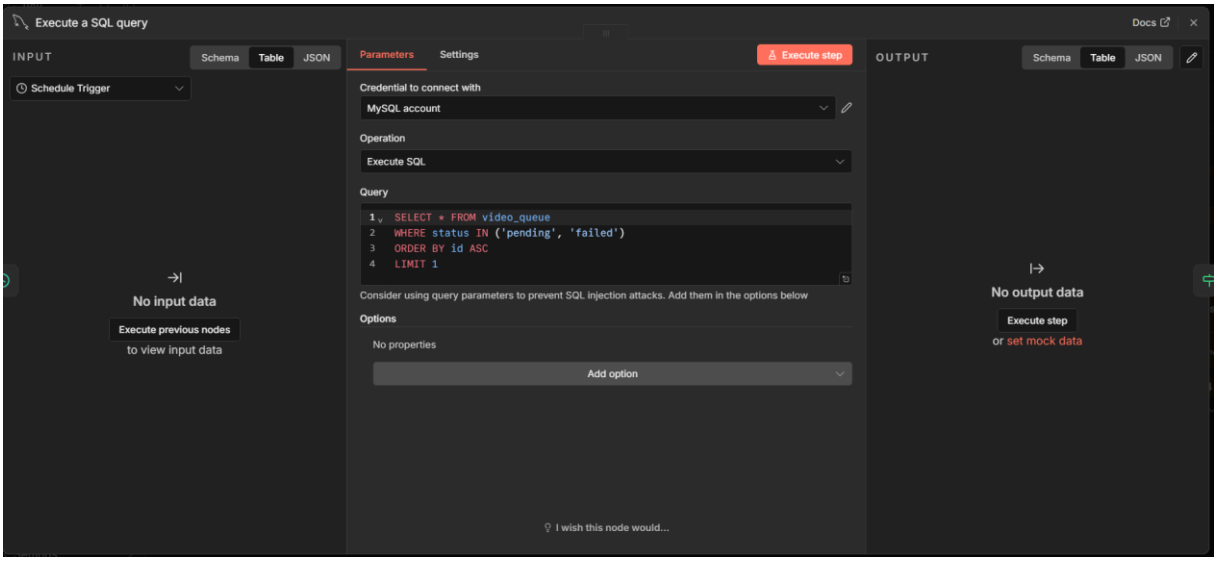
NODE 3 : Schedule Trigger

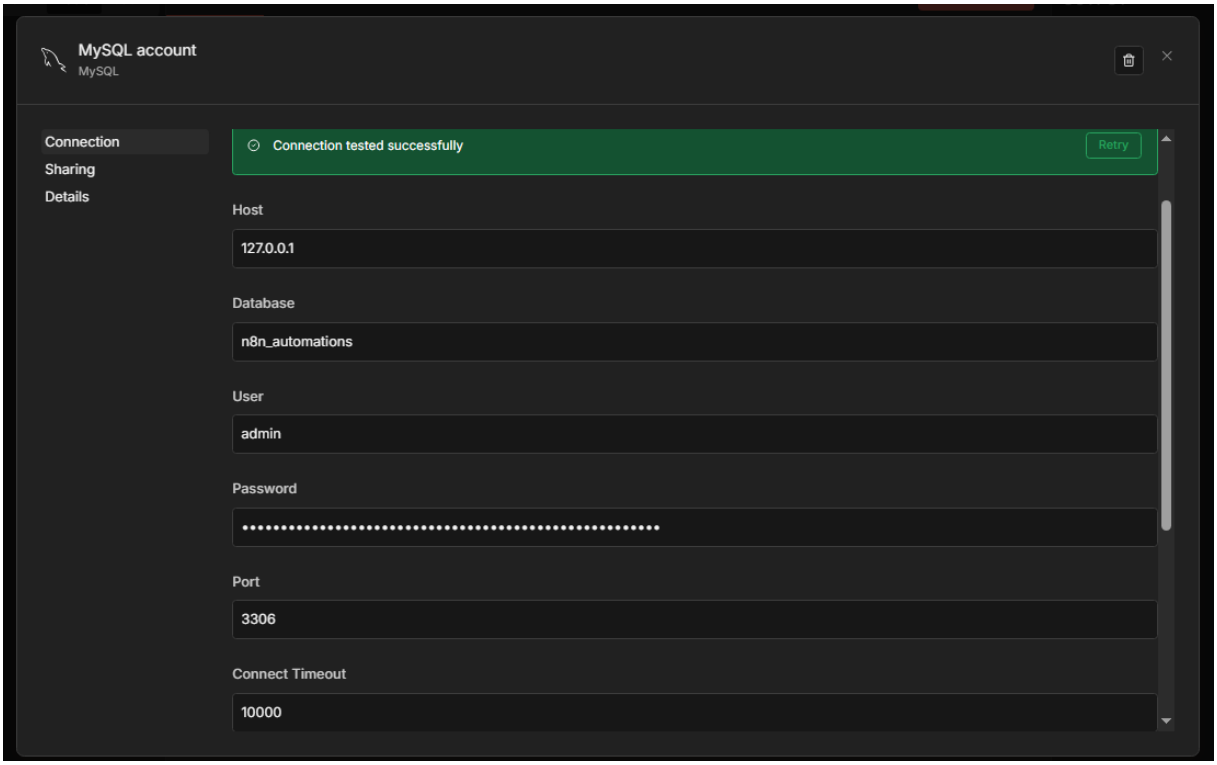


Explication du node :

- **Fonction :** Déclenche toutes les 20 minutes pour traiter la file automatiquement.
- **Paramètres clés :** interval: 20 minutes, disabled: true.
- **Entrée :** Rien.
- **Sortie :** 1 item vide.
- **Autres :** Désactivé → activez pour mode auto mais attention à l'interval pour éviter d'etre bloqué pour SPAM par l'API Tiktok.

NODE 4 : Execute a SQL query

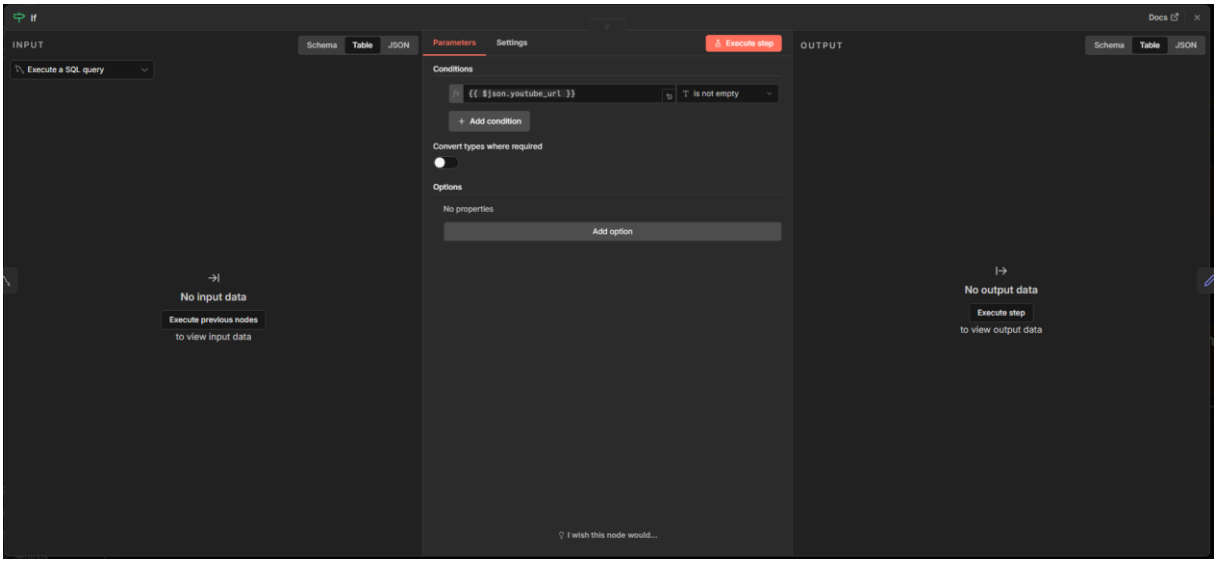




Explication du node :

- **Fonction :** Sélectionne 1 vidéo en attente/échouée (cron mode).
- **Paramètres clés :** Query SELECT * FROM video_queue WHERE status IN ('pending','failed') ORDER BY id ASC LIMIT 1.
- **Entrée :** Item du Schedule.
- **Sortie :** 1 ligne DB (ou 0).
- **Autres :** Credentials MySQL "MySQL account". ← à configurer en cliquant sur le crayon

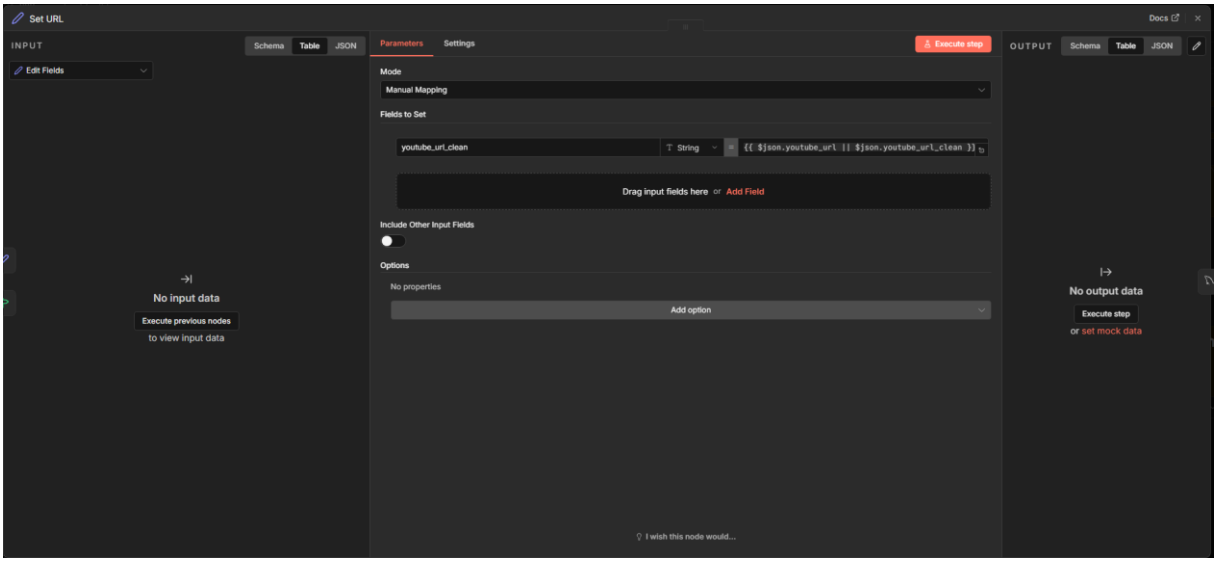
NODE 5 : If



Explication du node :

- **Fonction :** Vérifie URL non vide.
- **Paramètres clés :** Condition youtube_url notEmpty (strict).
- **Entrée :** Ligne DB.
- **Sortie :** true (valide) ou false (skip).
- **Autres :** caseSensitive: true.

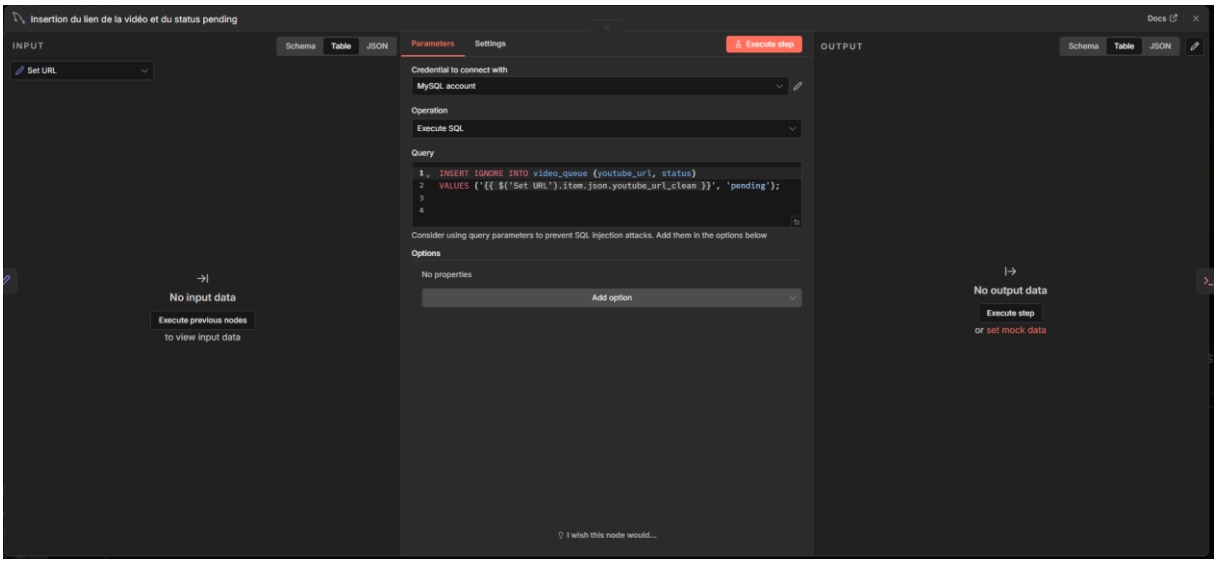
NODE 6 : Set URL



Explication du node :

- **Fonction :** Normalise l'URL.
- **Paramètres clés :** youtube_url_clean = $json.youtube_url || $json.youtube_url_clean.
- **Entrée :** Ligne DB ou webhook.
- **Sortie :** Item normalisé.
- **Autres :** Gère les 2 chemins (cron/webhook).

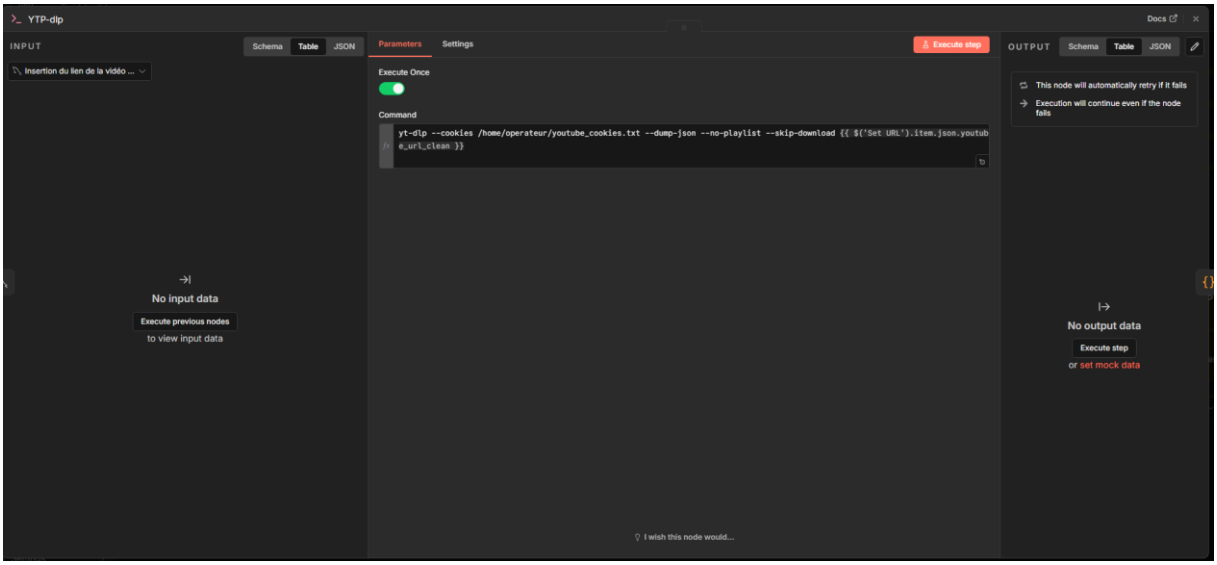
NODE 7 : Insertion du lien de la vidéo et du status pending



Explication du node :

- **Fonction :** Ajoute l'URL en file DB.
- **Paramètres clés :** INSERT IGNORE INTO video_queue (youtube_url, status) VALUES (...).
- **Entrée :** URL clean.
- **Sortie :** Résultat INSERT.
- **Autres :** IGNORE = skip si existe déjà.

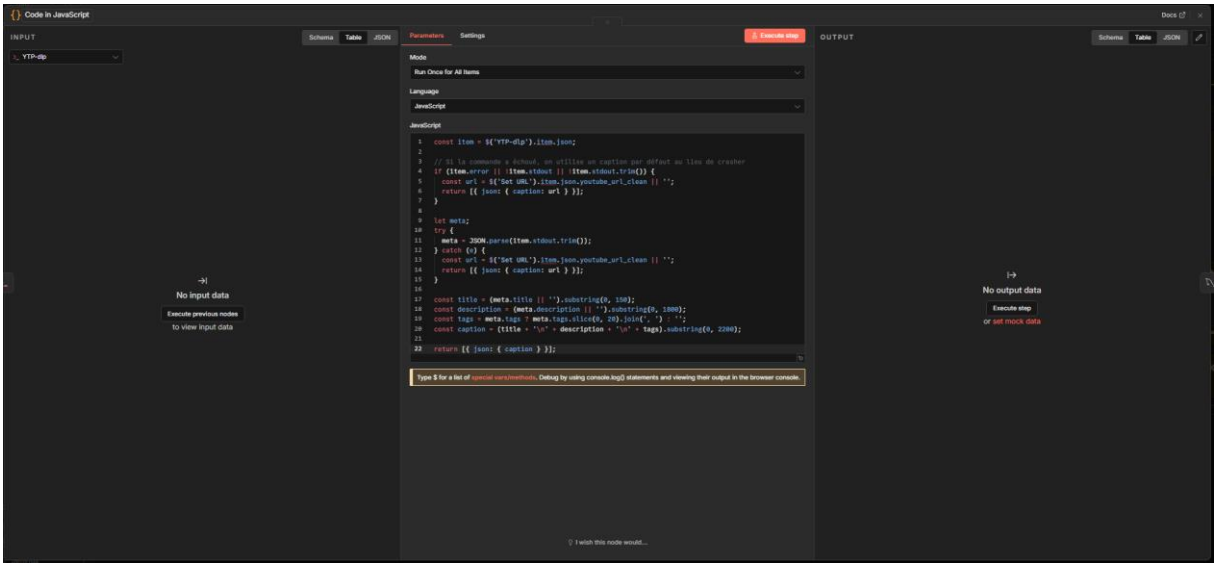
NODE 8 : YTP-dlp



Explication du node :

- **Fonction :** Récupère metadata JSON YouTube (sans download).
- **Paramètres clés :** yt-dlp --dump-json --cookies ... --skip-download {{youtube_url_clean}}.
- **Entrée :** URL clean.
- **Sortie :** stdout = JSON metadata.
- **Autres :** retryOnFail=true, onError=continueRegularOutput.

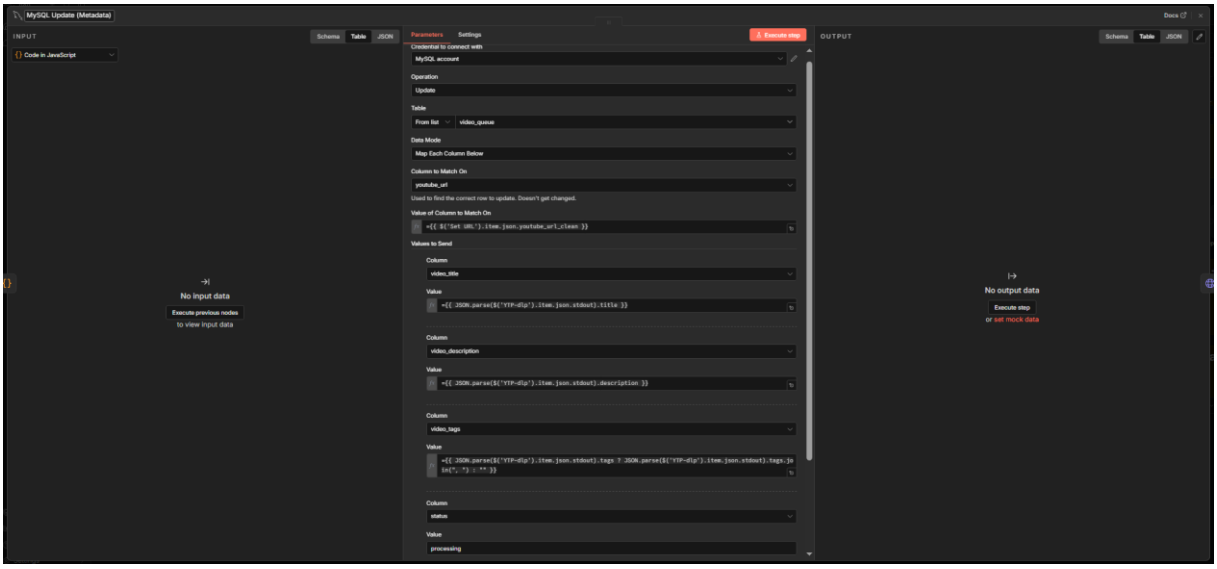
NODE 9 : Code in JavaScript



Explication du node :

- **Fonction :** Parse metadata → génère caption TikTok.
- **Paramètres clés :** JS code (try/catch + truncate + fallback URL).
- **Entrée :** stdout yt-dlp.
- **Sortie :** 1 item {caption} (≤2200 chars).
- **Autres :** Robuste aux erreurs yt-dlp.

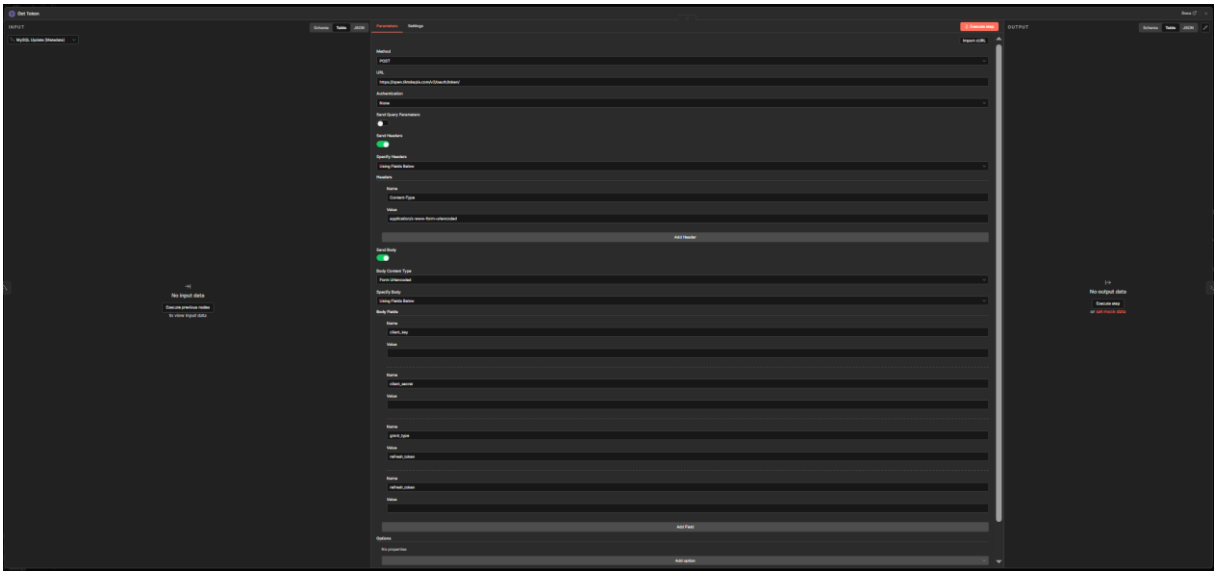
NODE 10 : MySQL Update (Metadata)



Explication du node :

- **Fonction :** Stocke title/desc/tags + status=processing.
- **Paramètres clés :** Update video_queue WHERE youtube_url, set title/desc/tags/status.
- **Entrée :** Caption + metadata.
- **Sortie :** Résultat update.
- **Autres :** Parse JSON dans expressions {{JSON.parse(…)}}.

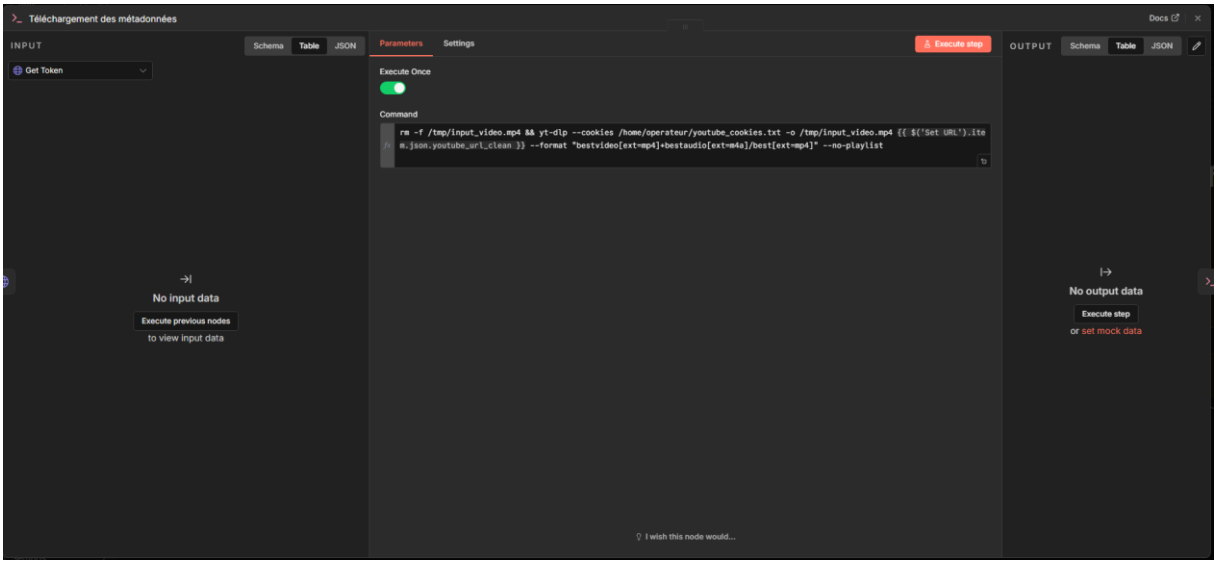
NODE 11 : Get Token



Explication du node :

- **Fonction :** Refresh access_token.
- **Paramètres clés :** POST /v2/oauth/token/ form grant_type=refresh_token.
- **Entrée :** Après metadata DB.
- **Sortie :** {access_token} (24h validité).
- **Autres :** Credentials client_key/secret/refresh_token à setup.

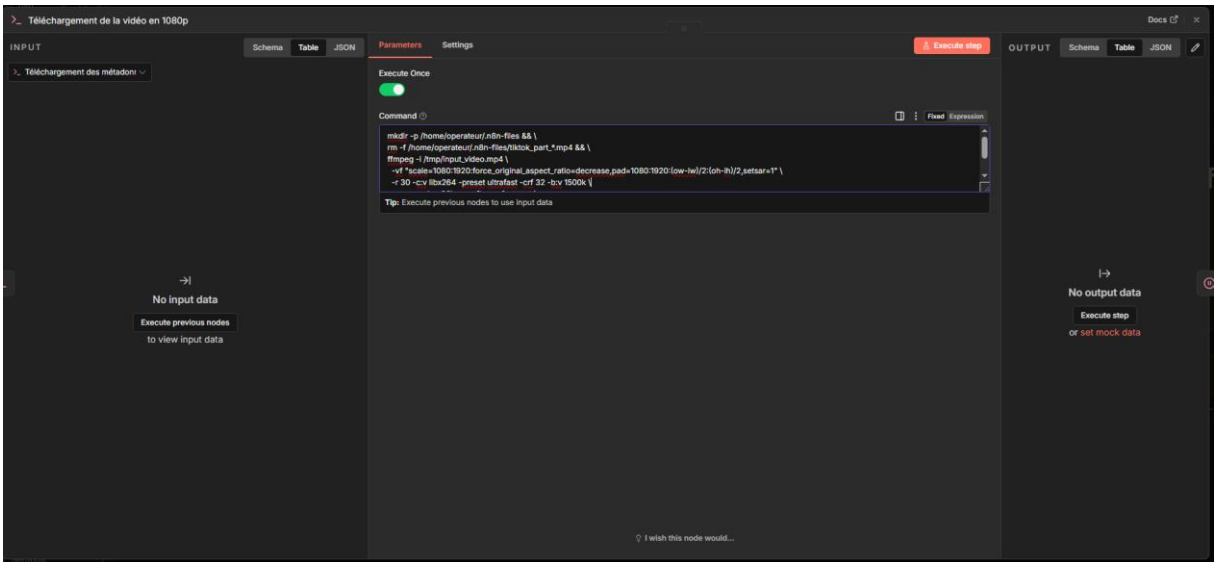
NODE 12 : Téléchargement des métadonnées



Explication du node :

- **Fonction :** Télécharge la vidéo source.
- **Paramètres clés :** yt-dlp -o /tmp/input_video.mp4 --format bestvideo [ext=mp4]+bestaudio[ext=m4a]/best[ext=mp4]...
- **Entrée :** Après token.
- **Sortie :** stdout vide si OK.

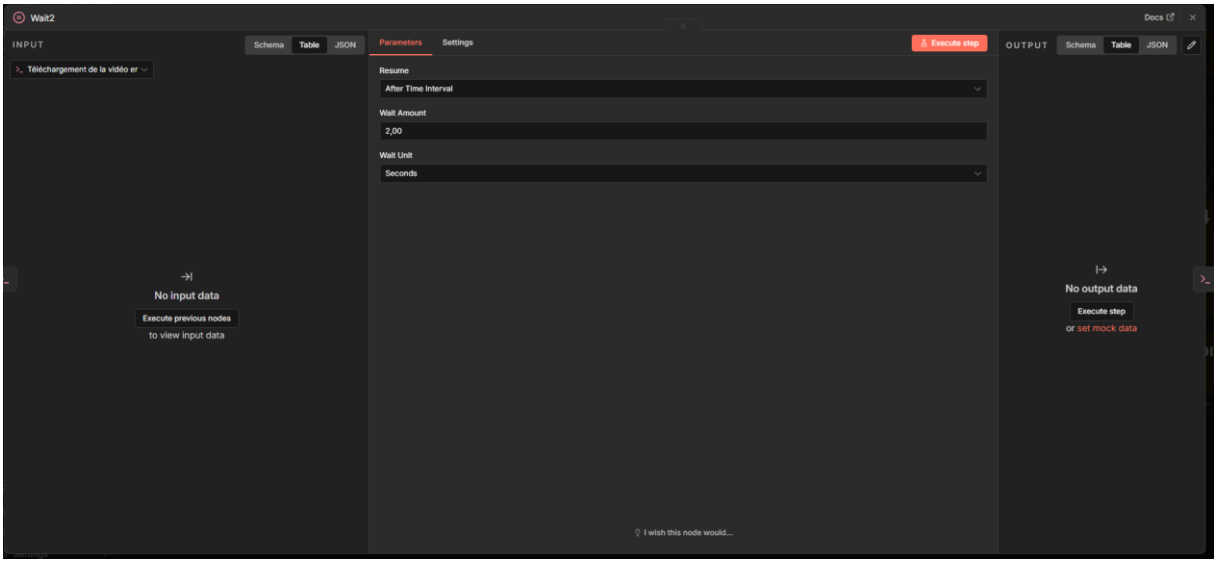
NODE 13 : Téléchargement de la vidéo en 1080p



Explication du node :

- **Fonction :** Transcode vertical + segmente 90s/partie.
- **Paramètres clés :** ffmpeg scale/pad 1080x1920, H264/AAC, -segment_time 90.
- **Entrée :** Vidéo /tmp.
- **Sortie :** stdout ffmpeg.
- **Autres :** Output → /home/operateur/.n8n-files/tiktok_part_*.mp4.

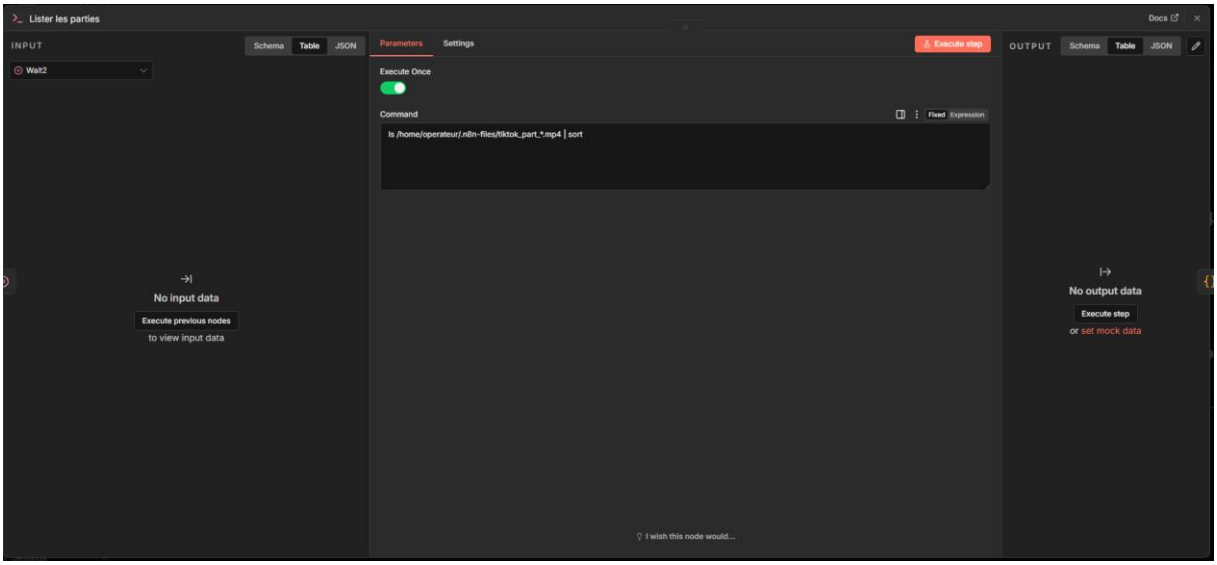
NODE 14 : Wait2



Explication du node :

- **Fonction :** Pause 2s (fsync).
- **Paramètres clés :** amount: 2 (secondes).
- **Entrée/Sortie :** Passe l'item.

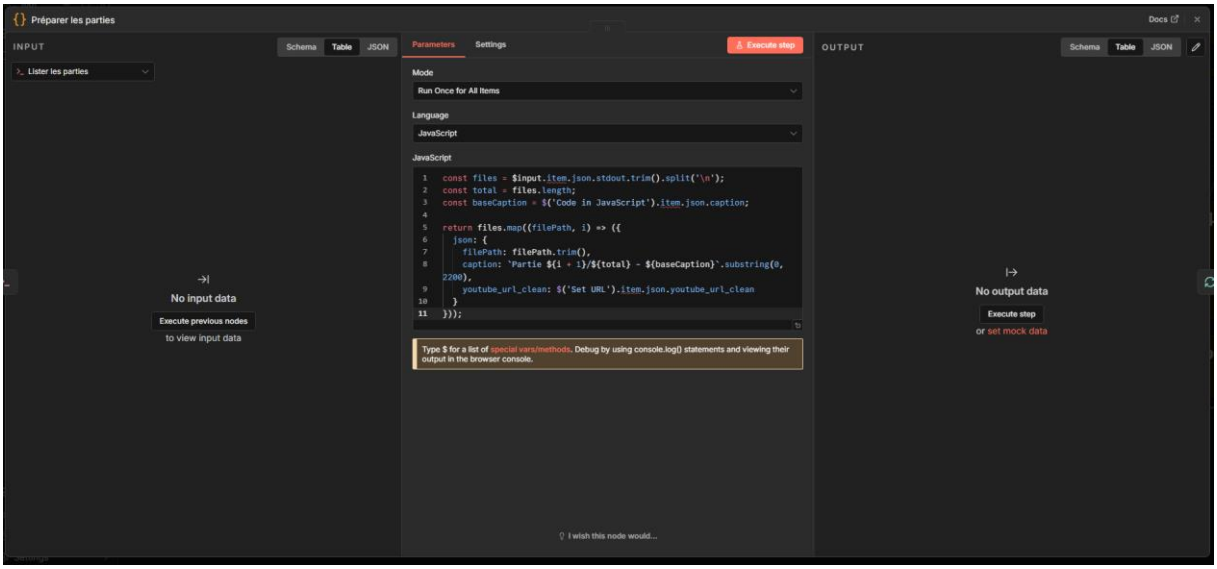
NODE 15 : Lister les parties



Explication du node :

- **Fonction :** Liste fichiers segments.
- **Paramètres clés :** ls … | sort.
- **Entrée :** Après ffmpeg.
- **Sortie :** stdout = liste chemins.

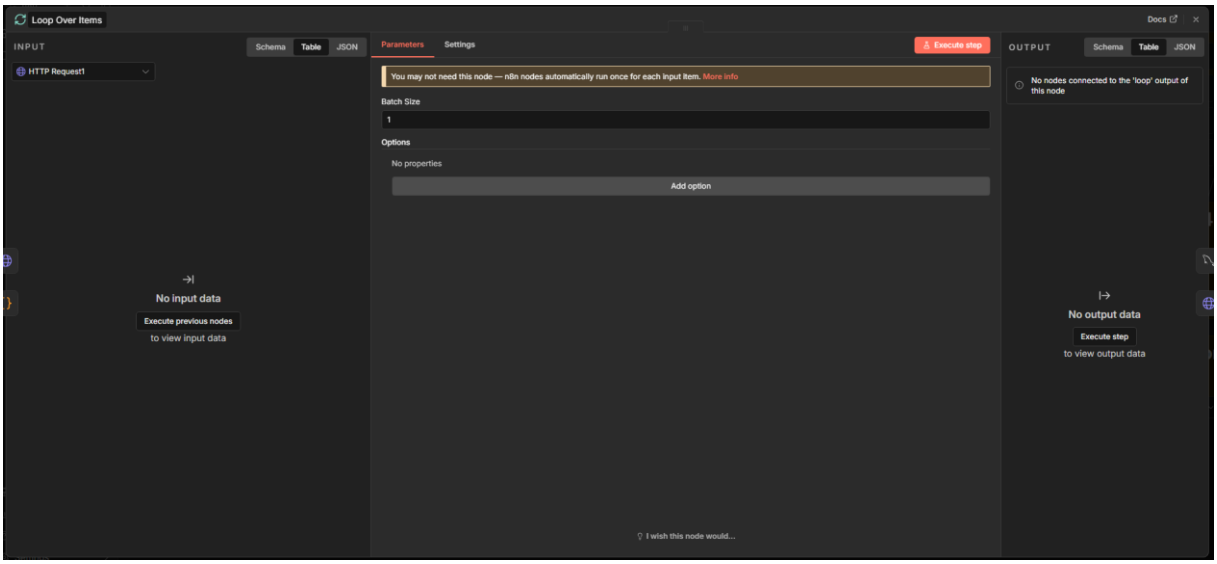
NODE 16 : Préparer les parties



Explication du node :

- **Fonction :** Crée 1 item par segment.
- **Paramètres clés :** JS split() + map {filePath, caption="Partie X/N", youtube_url_clean}.
- **Entrée :** Liste fichiers.
- **Sortie :** N items (1/segment).

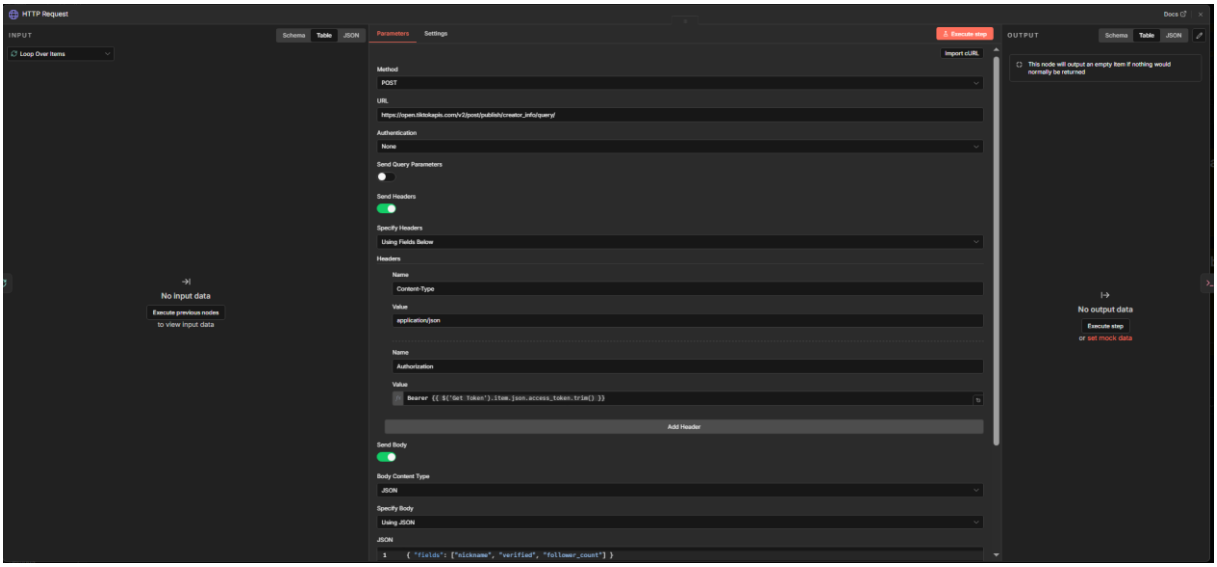
NODE 17 : Loop Over Items



Explication du node :

- **Fonction :** Boucle sur segments (batch=1).
- **Paramètres clés :** amount: 1.
- **Entrée :** N items segments.
- **Sortie :** Item courant (0) ou fin boucle (1).

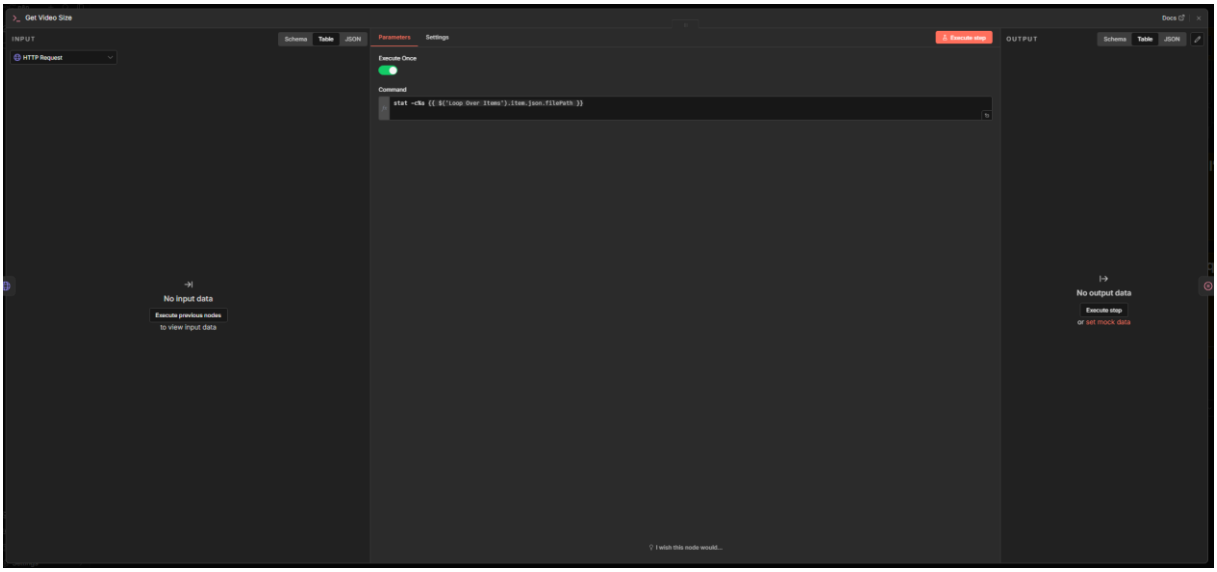
NODE 18 : HTTP Request



Explication du node :

- **Fonction :** Récupère infos compte.
- **Paramètres clés :** POST /v2/post/publish/creator_info/query/ {"fields": ["nickname","verified","follower_count"]}.
- **Entrée :** Après token.
- **Sortie :** Infos creator.

NODE 19 : Get Video Size



Explication du node :

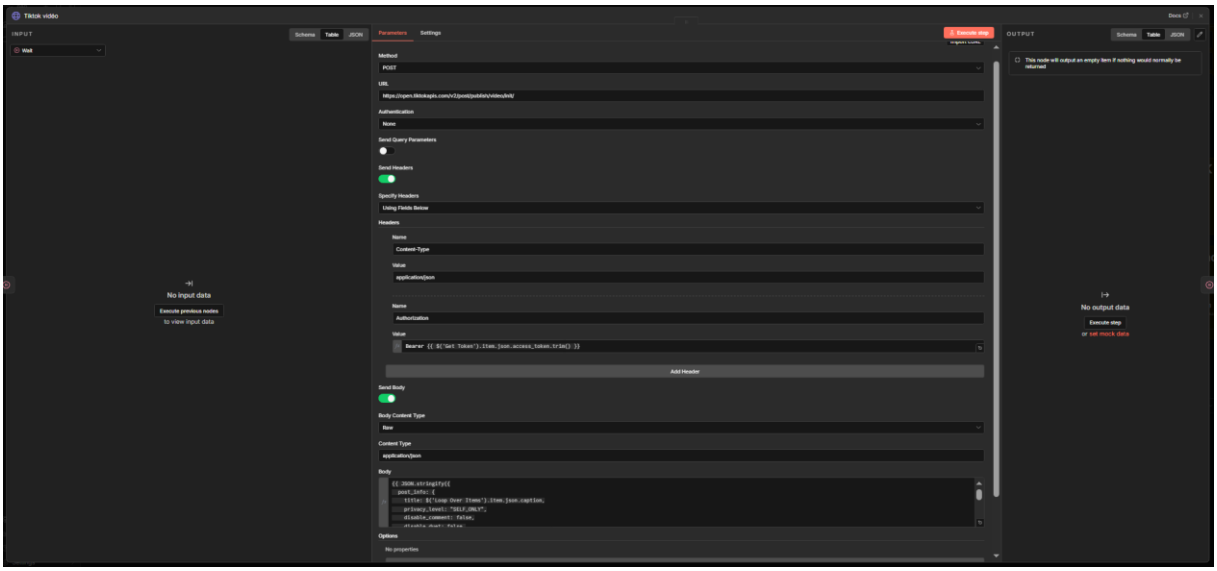
- **Fonction :** Taille fichier pour Content-Range.
- **Paramètres clés :** stat -c%s {{filePath}}.
- **Entrée :** Item segment.
- **Sortie :** stdout = bytes.

NODES 20 : Wait / Wait3 / Wait1 / Wait4

Explication du node :

- **Fonction :** Pauses (1s, 2s, 30s, 2s).
- **Paramètres clés :** amount: X secondes.
- **Autres :** Timing → Pour éviter le SPAM de requêtes

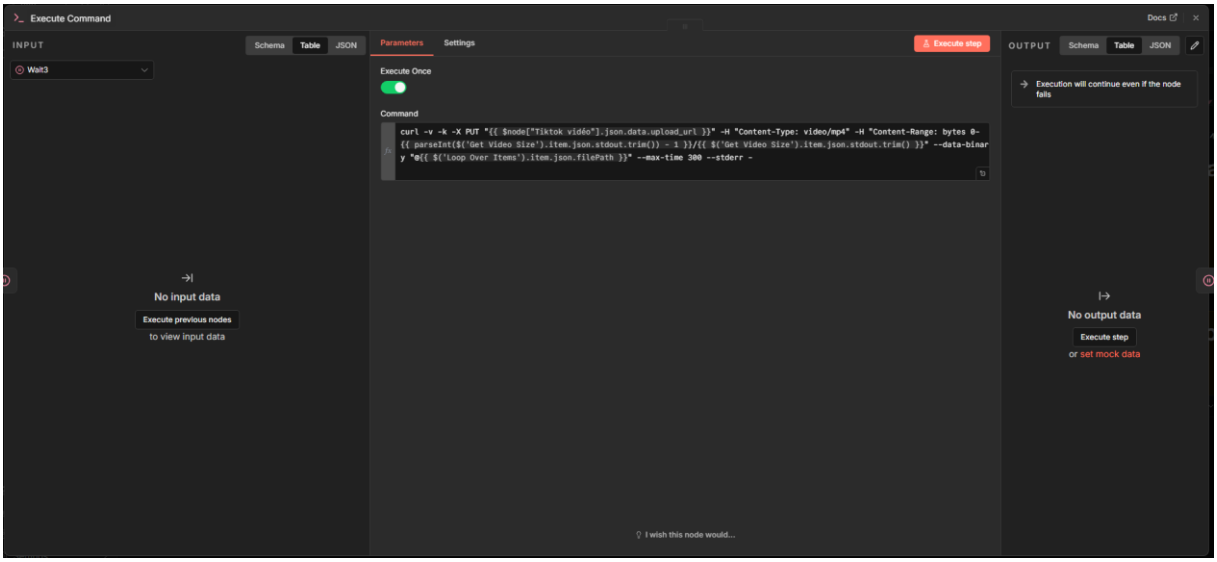
NODES 21 : Tiktok vidéo



Explication du node :

- **Fonction :** Initialise upload TikTok.
- **Paramètres clés :** POST /v2/post/publish/video/init/ JSON post_info + source_info (size/chunk=1).
- **Entrée :** Size + caption.
- **Sortie :** {data: {upload_url, publish_id}}.
- **Autres:** privacy_level: "SELF_ONLY" = Mode Sandbox, privacy_level:"PUBLIC" = Mode production donc il faut absolument le modifier dans le JSON du node

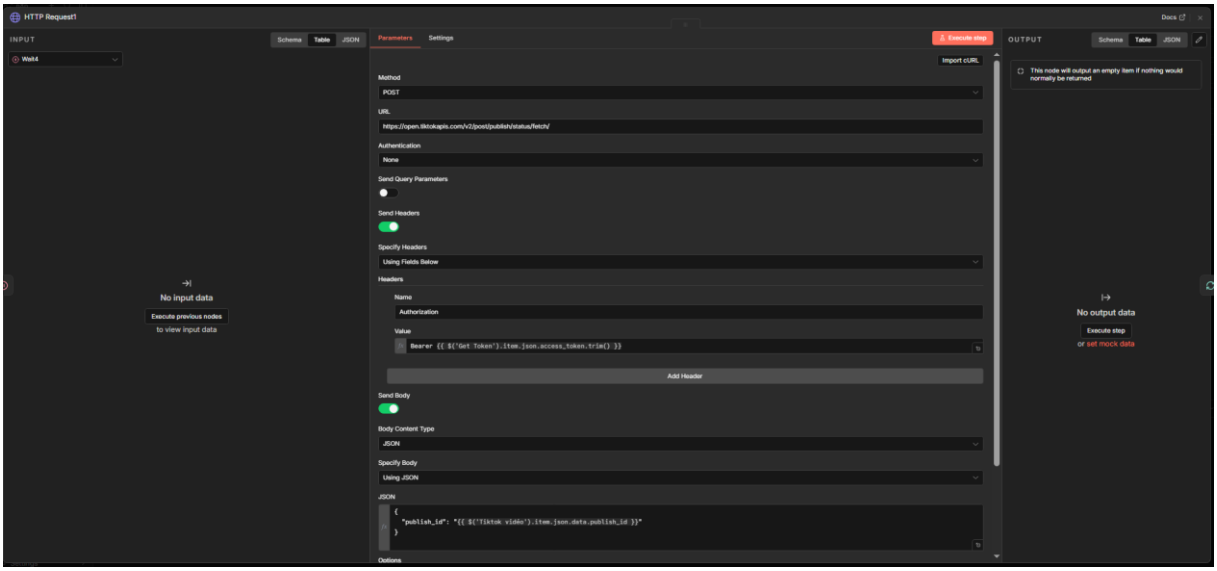
NODE 22 : Execute Command (upload)



Explication du node :

- **Fonction :** Upload PUT vers upload_url.
- **Paramètres clés :** curl -k PUT "{{upload_url}}" Content-Range --data-binary "@filePath".
- **Entrée :** upload_url.
- **Sortie :** stdout curl (onError: continue).

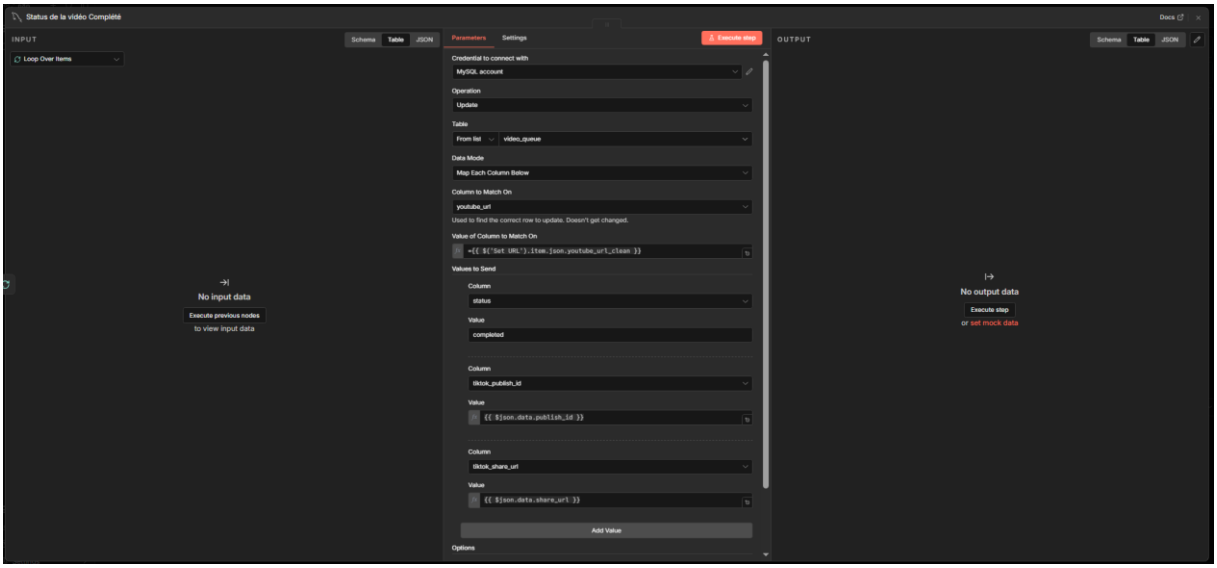
NODE 23 : HTTP Request1



Explication du node :

- **Fonction :** Vérifie status publication.
- **Paramètres clés :** POST /v2/post/publish/status/fetch/ {"publish_id"}.
- **Entrée :** Après upload.
- **Sortie :** Status + share_url.

NODE 24 : Status de la vidéo complétée



Explication du node :

- **Fonction :** Marque fin + stocke résultats.
- **Paramètres clés :** Update status='completed', tiktok_publish_id, tiktok_share_url.
- **Entrée :** Status TikTok.
- **Sortie :** Résultat update.
- **Autres :** Appelée par segment → dernier overwrite les autres !